

A Survey of Change Point Detection in Dynamic Graphs

Yuxuan Zhou , Shang Gao , Dandan Guo , Xiaohui Wei , Jon Rokne , *Life Fellow, IEEE*, and Hui Wang 

(Survey Paper)

Abstract—Change point detection is crucial for identifying state transitions and anomalies in dynamic systems, with applications in network security, health care, and social network analysis. Dynamic systems are represented by dynamic graphs with spatial and temporal dimensions. As objects and their relations in a dynamic graph change over time, detecting these changes is essential. Numerous methods for change point detection in dynamic graphs have been developed, but no systematic review exists. This paper addresses this gap by introducing change point detection tasks in dynamic graphs, discussing two tasks based on input data types: detection in graph snapshot series (focusing on graph topology changes) and time series on graphs (focusing on changes in graph entities with temporal dynamics). We then present related challenges and applications, provide a comprehensive taxonomy of surveyed methods, including datasets and evaluation metrics, and discuss promising research directions.

Index Terms—Dynamic graphs, time series on graphs, change point detection, anomaly detection.

I. INTRODUCTION

GRAPHS are powerful tools for modeling real-world systems, capturing relationships among entities and their connections [1], [2]. They are crucial in domains like social networks, biological interactions, and information propagation. In social networks, nodes represent individuals and edges denote relationships, aiding the study of information diffusion and community structures [3], [4], [5]. In neural networks, node represent brain regions and edges denote functional connections [6].

Dynamic systems evolve due to events, and their graph models, called *dynamic graphs* [7], evolve too. This paper focuses

Received 27 December 2023; revised 4 November 2024; accepted 23 December 2024. Date of publication 27 December 2024; date of current version 5 February 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62272190 and Grant 62306125, in part by the National Key Research and Development Program of China under Grant 2023YFB4502304, in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) under Grant 7222, and in part by the UK Engineering and Physical Science Research Council under Grant EP/V002740/2 and Grant EP/V026488/2. Recommended for acceptance by X. Yu. (Corresponding author: Shang Gao.)

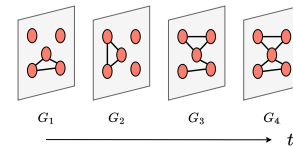
Yuxuan Zhou, Shang Gao, and Xiaohui Wei are with the College of Computer Science and Technology, Jilin University, Changchun 130012, China (e-mail: shanggao@jlu.edu.cn).

Dandan Guo is with the School of Artificial Intelligence, Jilin University, Changchun 130012, China.

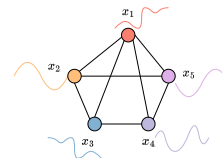
Jon Rokne is with the Department of Computer Science, University of Calgary, Calgary, AB T2N 1N4, Canada.

Hui Wang is with the School of Electronics, Electrical Engineering and Computer Science, Queen's University, BT7 1NN Belfast, U.K.

Digital Object Identifier 10.1109/TKDE.2024.3523857



(a) A graph snapshot series reflects topological evolution of a graph.

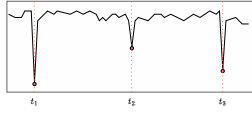


(b) Time series on graphs entail reflect temporal dynamics in entities.

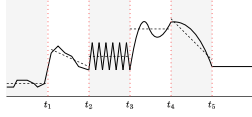
Fig. 1. A comparison between the graph snapshot series and the time series on graphs.

on two dynamic graph models: graph snapshot series and time series on graphs [7]. A graph snapshot series is a sequence of graphs representing network topology at different times, capturing nodes and edges at specific moments, and allowing for the analysis of network topology changes over time [7], [8], [9]. For example, social network snapshots show evolving friendships, helping detect emerging trends and understand social dynamics [10]. Time series on graphs involve time series organized into graphs, monitoring numerical attributes of nodes or edges over time. For example, sensor networks in smart cities measure environmental conditions, helping detect changes and understand their spatial propagation mechanisms. Fig. 1 illustrates the difference between these two dynamic graphs.

Change point detection in dynamic graphs is essential for extracting insights from evolving systems. It enables real-time detection of emerging patterns or trends [11], valuable in applications like smart cities [11], healthcare [12], and cybersecurity [13]. This allows timely responses to changing conditions. By identifying significant changes in graph topology or attributes, researchers can understand the processes driving system dynamics [14]. It also aids in developing adaptive and resilient systems. Knowing when and where changes occur helps systems adapt and enhance their resilience to unforeseen circumstances [14]. For example, in financial networks, changes in connectivity patterns can indicate fraudulent activity. In computer networks, detecting abrupt changes in



(a) Anomalies are rare events that may not change the data distribution.



(b) Change points are times with significant shifts in data distribution.

Fig. 2. A comparison between anomalies and change points.

data flow can identify cyber-attacks, enabling rapid mitigation actions.

A formal definition of change point detection task in dynamic graphs is given as follows. Given a sequence of observations $\{\mathbf{G}_i\}$ representing dynamic graphs, which may be composed of a graph snapshot series or time series on graphs, the change point detection task can be formulated as a hypothesis testing problem with null hypothesis

$$H_0 : \mathbf{G}_i \sim \mathbb{P}_0, \quad i = 1, \dots, m$$

against the single change point alternative

$$H_1 : \exists 1 \leq t < m, \quad \mathbf{G}_i \sim \begin{cases} \mathbb{P}_1, & i > t \\ \mathbb{P}_0, & \text{otherwise} \end{cases}$$

or the changed interval alternative

$$H_2 : \exists 1 \leq t_1 < t_2 \leq m, \quad \mathbf{G}_i \sim \begin{cases} \mathbb{P}_1, & i = t_1 + 1, \dots, t_2 \\ \mathbb{P}_0, & \text{otherwise} \end{cases}$$

where $\mathbb{P}_0$ and $\mathbb{P}_1$ are two probability measures that differ on a set of non-zero measures [15].

Related concepts to change point detection include anomaly detection and novelty detection. Anomalies are patterns that deviate from normal behavior [16]. Anomaly detection finds unusual phenomena in time series (Fig. 2(a)), while change detection identifies time points of significant changes (Fig. 2(b)). Changes alter the statistical distribution of observations, unlike anomalies [17]. Despite differences, algorithms like CUSUM and EWMA detect both changes and anomalies. Novelty detection finds test data points differing from training data, representing something new [18]. Concept drift in tensors is another related concept [19], [20], [21], [22], [23], [24]. It refers to changes in the relationship between input data and target variables in online supervised learning [19]. Concept drift (CD) and change point detection (CPD) differ in change types and objectives. CD addresses sudden, incremental, gradual, and recurring changes [22], while CPD focuses on sudden shifts between variable states [25]. Regarding research objectives, CD focuses on examining the relationship between input features and the target variable [26], identifying and adapting to continuous changes in data distribution [23] to ensure machine learning models remain accurate over time [24], while CPD mainly concentrates

on pinpointing specific moments of abrupt changes in data properties [27]. In the scenario to detect changes in dynamic graphs, CPD aims to find out abrupt changes in topology or global attributes over time.

Identifying changes in dynamic graphs typically involves graph representation and time series change detection, which together complement the change point detection process. Graph representation, or graph embedding, represents a graph as low-dimensional vectors while preserving graph structures [1]. For dynamic graphs, the aim is to discover frequent “graph patterns” over time. These patterns, represented as feature vectors, matrices, or fused scalars, encapsulate graph snapshots or time series with minimal noise. Change points are detected when these patterns undergo significant changes. In [28], three popular frameworks for anomaly detection in time series, extendable to change point detection in dynamic graphs, are summarized. First, graph representation and traditional time series change detection can act as separate upstream and downstream steps. Second, they can form an end-to-end framework, commonly used in deep learning-based methods. Third, some frameworks directly mine features before and after change points, detecting changes through pattern matching or classification. These frameworks are illustrated in Fig. 3 using graph snapshot series. These frameworks are also applicable to time series on graphs with appropriate preprocessing.

Over the past 20 years, dynamic graph change point detection techniques have been developed in various application domains, including network security [13], [29], [30], [31], control science [32], and neuroscience [33], [34], [35]. Early methods used statistical theories like the Auto-regressive model (AR) [36] and the Cumulative Sum (CUSUM) algorithm [4], [13], [37], [38]. Machine learning approaches such as Support Vector Machine (SVM) [39], [40] were later applied. Recently, deep learning methods utilizing graph neural networks (GNN) [41], [42], long-short-term-memory (LSTM) frameworks [30], [43], and Generative Adversarial Networks (GAN) [44] have also been developed for CPD in dynamic graphs.

This survey systematically reviews change point detection in dynamic graphs, covering challenges, applications, methods, datasets, evaluation metrics, and future directions. Specifically, we offer:

- 1) *A new perspective*: Our survey is the first to formally distinguish two types of dynamic graphs, bridging gaps in existing surveys that focus on either graph topology changes or multivariate time series correlations.
- 2) *A comprehensive review*: We summarize common challenges and applications, and provide a thorough review of existing techniques, including their motivations and main ideas.
- 3) *A new taxonomy*: We classify dynamic graph types and change point detection methods into statistical, data mining, and deep learning classes, as shown in Table I.

The rest of paper is structured as follows: Section II discusses related domains and surveys. Section III summarizes common challenges in dynamic graph change point detection. Section IV reviews applications in network security, IoT monitoring, neuroscience, and industrial control. Section V details existing

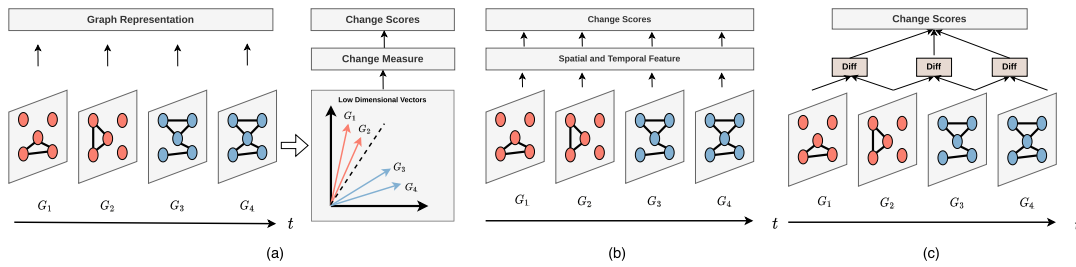


Fig. 3. Frameworks for detecting change points in dynamic graphs using graph snapshots. A change occurs in connection density between G_2 and G_3 . (a) Two-step approach: extract low-dimensional representation from each snapshot and send to change point detector. Representation and detection are upstream and downstream in a pipeline; (b) End-to-end approach: Change detector extracts features and outputs change score directly for each snapshot; (c) Time series approach: learn features before and after change points from graph snapshots, then use a classifier to output change scores.

TABLE I
TAXONOMY OF SURVEYED WORKS

Type of Dynamic Graph	Method Class	Techniques
Graph Snapshot Series	Attribute Analysis	Attribute Fusion [36], [45], [46]
		Scan Statistic Analysis [47]
	Matrix Factorization	Eigenvector-based Methods [11], [48], [49]
		Graph-operator-based Methods [4]
		Graph-signal-processing-based Methods [50], [51]
	Probabilistic Methods	Markov Modeling [52]
		Stochastic Modeling [53], [54], [55], [56]
	Sampling	Random-walk-based Methods [57]
		Graph-sketch-based Methods [58]
	Time Series on Graphs	Community Analysis [5], [10], [59], [60]
Compression Cost Analysis [3], [61], [62], [63]		
Graph Kernels [40], [64], [65]		
Reconstruction Residual Analysis [66], [67]		
Direct Detection [37], [43], [68]		
Dimension Reduction [69], [70], [71]		
Correlation Analysis [29], [72], [73], [74]		
Entropy Analysis [75], [76], [77]		
Deep Learning		RNN-based Methods [32]
		Autoencoder-based Methods [39], [42], [78], [79], [80]
	GAN-based Methods [44]	

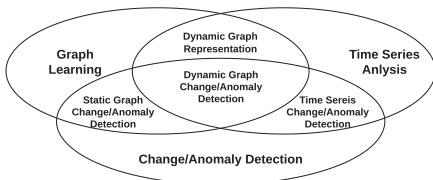


Fig. 4. Interconnections between research topics relevant to our survey.

works on change point detection in graph snapshot series and time series on graphs. Section VI reviews available datasets and evaluation metrics. Section VII concludes the survey and suggests future research directions.

II. RELATED WORKS

Change point detection in dynamic graphs intersects graph learning and time series change detection (shown in Fig. 4). This section reviews related areas and surveys.

Time series change and anomaly detection: This well-studied topic aims to find state transitions or anomalous patterns in time series. We introduce it from two perspectives: data acquisition methods and assumptions on data distributions. Based on data acquisition, detection methods are classified into offline [27] and online [4]. Offline detection assumes the whole data sequence

is available, akin to time series segmentation. Online detection handles data sequentially, detecting change points based on current data. Offline methods can be adapted for online use with techniques like sliding time windows [13]. Since real-world applications rarely have the entire data sequence upfront, online methods are often “trained” offline and then deployed. Detection methods are also categorized into parametric and nonparametric based on data distribution assumptions. Parametric methods assume specific distributions (e.g., Gaussian), while nonparametric methods do not.

Commonly researched change point detection methods include likelihood-test-based [13], [50], [55], [56], [81], [82], Bayesian-based [53], [83], prediction-based [36], [43], classification-based [32], clustering-based [84], and graph-based methods [15], [85]. Several comprehensive surveys on time series change and anomaly detection have been published. Online change point detection techniques are reviewed in [25], and offline segmentation techniques, including cost functions and research methods, are detailed in [27]. Deep-learning-based frameworks are covered in [28]. Anomaly detection in IoT applications is reviewed for both univariate and multivariate time series [86].

Graph representation: Graph representation (or embedding) aims to represent graphs as low-dimensional vectors while preserving structures [1], reducing data complexity and noise for

TABLE II
SUMMARY OF RELATED SURVEYS

Research Domain	Surveys	AD	CPD	GAD	GSAD	GSCPD	GTSAD	GTSCPD
Time Series Change/Anomaly Detection	Chandola et.al. [16]	✓						
	Wang et.al. [95]	✓						
	Cook et.al. [86]	✓						
	Pang et.al. [28]	✓						
	Truong et.al. [27]		✓					
	Aminikhanghahi et.al. [25]		✓					
Graph Change/Anomaly Detection	Akogulu et.al. [89]			✓	✓	✓		
	Ma et.al. [90]			✓	✓	✓		
	Ranshous et.al. [91]				✓	✓		
	Savage et.al. [92]			✓	✓	✓		
	Pazho et.al. [93]			✓	✓	✓		
	Jin et.al. [94]			✓	✓			
	our survey				✓	✓	✓	✓

CPD: Change point detection. GAD: Anomaly detection in static graphs. GSAD: Anomaly detection in graph snapshot series. GSCPD: Change point detection in graph snapshot series. GTSAD: Anomaly detection in time series on graphs. GTSCPD: Change point detection in time series on graphs.

downstream tasks. Numerous surveys on static and dynamic graph representation exist. For static graphs, reviews cover techniques like matrix factorization, graph kernels, and deep learning [1], [2], [87], [88]. For dynamic graphs, emerging techniques include temporal matrix factorization and dynamic graph autoencoders [7], [8].

Anomaly detection in graphs: Anomaly detection in graphs aims to find unusual substructures in static or dynamic graphs, focusing on graph topology anomalies. Reference [89] reviews anomaly detection challenges, techniques, and applications in both static and dynamic graphs. Reference [90] focuses on deep-learning-based techniques, while [91] categorizes anomalies in dynamic graphs into vertex, edge, and substructure anomalies, which we consider as topological changes in graph snapshot series. Domain-specific surveys, such as [92] for social networks and [93] for distributed systems, provide focused reviews. Reference [94] investigates the latest progress in Graph Neural Networks (GNN) for time series data (GNN4TS), summarizing applications in forecasting, classification, anomaly detection, and data imputation.

Existing works on change and anomaly detection in time series and graphs, summarized in Table II, rarely focus on change detection in dynamic graphs. This paper addresses this gap by introducing change point detection tasks in dynamic graphs.

III. CHALLENGES

Data complexity: Data intricacies, such as heterogeneity [96], impact graph model construction and change point detection accuracy. In social networks, various relationships and non-structured data like photos and videos add complexity. This additional information introduces challenges in graph representation [97]. Redundant user attributes can hinder the detection of real change points in community structures. Multimodal approaches, such as Graph Neural Networks (GNNs) and Multimodal Graph Attention Networks (MGAT) [97], show promise in effectively incorporating different data modalities. These methods partially address data heterogeneity by integrating diverse data types into unified models. However, scalability and computational efficiency remain challenges. Future research

could focus on improving these aspects for large-scale heterogeneous data.

Inter-dependence among graph nodes: Unlike univariate time series, graph nodes interact, so changes in one node can affect, be mitigated by, or be augmented by its neighbors [46]. This is common in graph substructures. To detect changes with low false alarm rates (FAR) [13], researchers often construct a global view of nodes rather than analyzing individual time series. However, this global perspective can make it difficult to pinpoint the exact source of changes. The challenge of node inter-dependence is partly addressed by methods combining global and local views, such as community detection and graph clustering [5], [36], [84]. These methods capture broader structural changes but still struggle with identifying the exact root causes. Future work could focus on enhancing causality detection and localized change detection at the subgraph or node level.

Noise: Noise in data, such as random variations, errors, or inconsistencies, affects data quality. In IoT networks, noise can arise from detector sensitivity variations, transmission errors, and other sources, making it hard to distinguish from true changes or anomalies. Noise may introduce misleading patterns or trends in change point detection in dynamic graphs. Some methods use robust statistical techniques and noise filtering to differentiate between noise and actual changes, but these often need domain-specific tuning and may not generalize well. Future research should focus on developing generalized, noise-robust algorithms adaptable to various types of noise and datasets.

Class imbalance: Changes and anomalies often make up a small portion of datasets. For example, in the KDD-99 intrusion detection dataset [98], anomalies are only 0.3% to 7.7% of the data. This imbalance challenges both supervised and unsupervised change point detectors, as the majority class can dominate the learning process, making it harder to recognize anomalies and changes [99], leading to high false alarm rates (FAR). Designing detectors resilient to class imbalance is a key challenge. To further address this challenge, one promising approach is to develop adaptive learning [21] algorithms that can dynamically adjust to the evolving class distribution in real-time. Additionally, leveraging synthetic data generation and transfer learning [100] from similar domains could enhance model robustness in the presence of class imbalance.

Lack of labels: In real-world scenarios, data often lacks class labels, making traditional supervised learning approaches infeasible since labeled data is essential for training and evaluating models [101]. Manually labeling change points is challenging due to practicality and cost issues, and furthermore the definition of change points varies across domains, complicating detection. Semi-supervised [102], [103] and unsupervised [104] learning methods show promise but are constrained by the quality and quantity of unlabeled data. Innovative techniques like active learning, where the model queries for labels on uncertain instances, and weak supervision, which utilizes noisy or limited labels, could offer more practical solutions.

Selection of change threshold: Most change point detection methods for dynamic graphs use a threshold to distinguish normal data from change points, which can be set manually or learned from historical datasets [43]. A threshold that's too large reduces precision, while one that's too small increases false alarm rates [13]. Dynamic thresholding techniques [13], [43] that adjust based on historical data and trends can be sensitive to parameter settings and inconsistent across scenarios. Research into more adaptive thresholding mechanisms could yield more reliable results.

Diverse causes of change point: The variety of internal and external factors leading to change points or anomalies in dynamic graphs makes it challenging to capture all possible behavior patterns [105]. For example, in a communication network, connection failures can result from physical modem errors, high router workload, or server DoS attacks. This diversity complicates the accurate detection of change points [14]. A comprehensive approach integrating multiple change detection methods [106] and causal inference techniques could address this challenge more effectively.

Domain-specific challenges: Data processing and modeling techniques vary across domains due to specific attributes [30], [32], [33], [35]. In intrusion detection, data consists of network flows with attributes like IPs and port numbers [75], posing challenges in identifying anomalous patterns. In neuroscience, data includes neuron potentials and functions [33], requiring techniques to extract meaningful patterns from complex, high-resolution neural signals.

IV. APPLICATIONS

A. CPD in Graph Snapshot Series

1) *Computer Network Security:* Computer networks, such as internet traffic and email networks, are generally modeled as dynamic graphs with nodes as devices/accounts and edges as connections. Change point detection (CPD) enhances network security by identifying sudden changes. For instance, increased activity among nodes may signal a spam attack, while a spike in connections could indicate a Distributed Denial of Service (DDoS) attack. Timely detection allows for prompt security measures. Reference [107] presents a framework for identifying spam accounts in evolving social networks using structural features and sequence modeling. Reference [108] tackles reviewing spam in e-commerce by modeling it as dynamic review networks and applying Clique Percolation Method (CPM) to rank reviewers based on spam indicators. Reference [109] introduces

Spatial-Temporal Graph Convolutional Networks (STGCN) for detecting and mitigating DDoS attacks in Software-Defined Networking (SDN). This approach uses STGCN to map the network and In-band Network Telemetry (INT) with adaptive sampling to identify switches under DDoS attack.

2) *Neural Network:* In neuroscience, neural networks are represented as dynamic graphs with nodes as brain regions and edges as functional connections from fMRI data. Change point detection (CPD) in these networks reveals interactions between brain regions and aids in intent analysis and diagnosing neurological disorders. Reference [6] and [110] use dynamic connectivity regression and random permutation to detect changes in neural activity from fMRI data. Reference [33] applies change point detection to resting state fMRI data using multiscale sliding windows and PCA, followed by local false discovery rate (FDR) analysis. Reference [35] detects transitions in BOLD time series with a latent block model and a Cumulative Sum (CUSUM)-based approach, using posterior predictive discrepancy (PPD) as an indicator. Reference [34] employs dynamic graph embedding and time clustering on Fourier-transformed BOLD time series, using spectral graph processing for change point detection.

3) *Social Network Analysis:* Social networks, including email, citation, and social media networks, can be modeled as dynamic graphs where nodes are entities and edges are interactions. Detecting changes in these graphs is crucial for understanding shifts in network structure and activity. For example, CPD can identify emerging research trends in citation networks, uncover communication pattern changes in communication networks, and track trending topics or unusual events in social media. Reference [10] proposes bit-wise definitions for detecting changes at both community and node levels. Reference [5] uses signal-processing-based CPD to monitor overlapping communities and structural changes. Reference [49] detects changes by monitoring eigenvector distances in graph snapshots with an exponentially weighted moving average (EWMA) control process.

4) *Reality Mining:* Reality mining analyzes sensor data to understand human behavior through dynamic graphs that represent interactions over time. CPD helps identify significant changes in interaction patterns, offering insights into social structure evolution and individual behavior changes in response to external events. For instance, change points can reveal the impact of new policies or societal events. In [111], the authors use the MIT Reality Mining dataset [112] to analyze individual and group behavior. Their multilinear clustering framework detects group dynamics and patterns of life from temporal sociographic data, demonstrating high accuracy in inferring identity from cell tower location sequences.

B. CPD in Time Series on Graphs

1) *Cybersecurity:* Cybersecurity involves monitoring network activities, represented as time series on graphs with nodes as entities and edges as interactions. CPD is crucial for identifying unusual workflows that may indicate malicious behavior. Research focuses on traffic flows and relationships between time series.

In [13], network traffic is grouped and analyzed using the non-parametric adaptive CUSUM (MNA-CUSUM) algorithm, which adjusts detection thresholds based on historical data. Subsequent work, such as [81], improves data summarization with Count-Min Sketch and uses multi-chart CUSUM (MNP-CUSUM) for change point detection. Later studies have analyzed correlations between time series for DoS attack detection, using Multivariate Correlation Analysis (MCA) as seen in [29], [72], and [73]. Machine learning methods are also employed for attack detection, utilizing techniques such as Restricted Boltzmann Machines (RBM) [113], Autoencoders, GANs [44], and various classification, regression, and clustering methods [78], [30].

2) *IoT Network Monitoring*: With the rise of IoT networks, monitoring for high availability, low energy consumption, and security is crucial. IoT networks, modeled as dynamic graphs, require change point and anomaly detection to identify unusual behavior.

Change point detection in multi-sensor networks can be centralized or decentralized. Centralized systems send data to a fusion center for global state analysis, using methods like CUSUM [114], [115]. Decentralized systems rely on sensor collaboration. In these systems, methods such as consensus algorithms [116] are used for detection, as demonstrated in [117] and [37]. To conserve energy, ECO-CUSUM [118] uses a power-saving approach where sensors exchange information only when anomalies are detected.

3) *Industry Control*: In industrial settings, production processes are monitored using sensor data on graphs, where nodes represent system parts and edges denote interactions. Change point detection (CPD) is essential for safety, identifying significant changes that signal potential hazards or inefficiencies, allowing timely interventions.

In [119], sensors attached to a structure use both univariate and multivariate CPD to detect structural flaws. In [120], sensors in industrial processes are grouped by minimum spanning trees, and a graph-based CPD technique [15] identifies state shifts. In [32], a multi-head CNN-RNN architecture detects anomalies in a service elevator monitored by sensors. In [121], change point detection algorithms are evaluated for predicting failures by monitoring short periods (mini-terms) to capture sudden changes as the Remaining Useful Life (RUL) nears its end, constructing industrial Big Data for failure prognostics. The study compares algorithms from [27] using 10 cases from the Mini-term 4.0 system.

4) *Traffic Monitoring*: Traffic networks are graph-based, with nodes as intersections or stations and edges as connecting roads or railways. Traffic data forms a time series on this graph, and change point detection (CPD) is essential for spotting congestion and anomalies. CPD helps identify significant deviations in traffic patterns, enabling timely management and improvement of traffic flow. In [122], a real-time congestion detection and reduction framework using streaming graph analytics is proposed. This framework uses a directed weighted graph to capture sensor data dependencies and detect traffic congestion. It includes online detection and congestion reduction mechanisms that identify critical areas for traffic light policy adjustments.

Reference [123] introduces a novel method for identifying traffic congestion warning communities to problems in heterogeneous urban networks. This method constructs a dynamic directed weighted network that incorporates link connectivity and real-time traffic loads. It uses Kalman filter-based predictions to adjust link weights and applies a three-step dynamic congestion warning detection process.

5) *Weather Forecasting*: Weather patterns can be analyzed with graphs, where nodes are geographical locations and edges represent meteorological connections. Change point detection (CPD) is crucial for identifying unusual weather patterns like those created by El-Nino. By monitoring meteorological data, CPD detects significant weather changes, aiding in early warning and preparation. In [124], a Graph Neural Network called Graphino is applied to seasonal climate forecasting, specifically for predicting the El Niño-Southern Oscillation (ENSO). The Graphino model, with its novel graph connectivity learning module, excels in capturing large-scale interactions and forecasting ENSO up to six months in advance.

6) *Health Care*: In health care, time series data on graphs—where nodes are geographical regions or health facilities and edges represent patient movements or disease transmission—are used to monitor outbreaks. Change point detection (CPD) identifies significant health data changes, enabling early outbreak detection and timely public health responses. In [12], network medicine highlights the complex relationships between diseases and molecular networks. Diseases often result from disruptions in these networks rather than single gene mutations. Identifying disease modules helps understand disease mechanisms and discover new therapeutic targets. In [125], mobile device data from Ebola outbreak areas is used to analyze network structure's impact on disease dynamics, leading to the development of Realistic Contact Networks (RCNs). RCNs model disease spread effectively by representing individuals as nodes and direct contacts as edges.

7) *Human Activity Analysis*: Human activities can be analyzed using time series data on graphs, where nodes are body parts or sensors and edges represent their interactions. Change point detection (CPD) in this context aids in gesture detection and other applications by identifying significant movement pattern changes, benefiting human-computer interaction, sports analysis, and rehabilitation. In [126], a novel hand gesture recognition method using Dynamic Graph-Based Spatial-Temporal Attention (DG-STA) is proposed. This method constructs a dynamic graph that adapts to each gesture, improving model flexibility and performance. The dynamic graph, built from hand skeleton data with nodes as joints and edges as connections, uses a self-attention mechanism in both spatial and temporal domains to learn edge weights and node features. This enables accurate capture of complex hand movements for gesture recognition.

V. CHANGE POINT DETECTION METHODS IN DYNAMIC GRAPH

A. Detection in Graph Snapshot Series

A graph snapshot series is a sequence of graphs sampled at regular intervals. Detecting change points in such a series focuses on analyzing the dynamic graph's topology. Various graph

TABLE III
SUMMARY OF SURVEYED WORKS FOR CHANGE POINT DETECTION IN GRAPH SNAPSHOT SERIES

Method Class	Reference	Dynamic Graph Model	Change Detection Method	Evaluation Metrics	Application Dataset
Attribute Analysis	Park et al. [45]	Graph Attribute Fusion	Hypothesis Test	Statistical Power	Enron
	Tang et al. [46]	Graph Attribute Fusion	Hypothesis Test	Statistical Power	Enron
	Shapnak et al. [51]	Scan statistics	GLRT	Statistical Power	Synthetic dataset
	Wang et al. [47]	Scan statistics	Hypothesis Test	Statistical Power	Enron
	Xie et al. [36]	Muti-layer Attribute Fusion	Permutation Test	Precision, Recall, F_1	World Trade
Matrix Factorization	Zhu et al. [84]	PageRank	Clustering	Precision, Recall, F_1	MIT, Legislative Cosponsorship
	Akoglu et al. [48]	SVD Factorization	Similarity Threshold	Precision, Recall	SMS network
	Hazrati et al. [49]	Eigenvectors	EWMA	Average Run Length	Communication Network
	Huang et al. [11]	SVD Factorization	Similarity Threshold	Hits at n	UCI Message, Legislative Cosponsorship Network
	Chen et al. [50]	Graph Signal Processing	CUSUM	Average Run Length	Canadian bill voting
Probabilistic Methods	Hwang et al. [4]	Graph Operator	Similarity Threshold	False Alarm Rate, Omitted Detection Rate, Delay	New York Taxi
	Wang et al. [52]	Markov Chain	Similarity Threshold	Precision, Recall	Synthetic dataset
	Peel et al. [53]	Hierarchical Random Graph Model	Bayesian	Precision, Recall	Legislative Cosponsorship Network
	Wilson et al. [54]	Stochastic Block Model	EWMA	Average Run Length	MIT, Enron
	Padilla et al. [55]	Random Dot Model	CUSUM	Absolute Error	Legislative Cosponsorship Network
Sampling-based Methods	Marengo et al. [56]	Random Dot Model	CUSUM	Detection Delay	Wireless Network Data, MIT, South American Football Matches
	Yu et al. [57]	Random Walk	Online K-means	AUC	Message
	Salem et al. [81]	Count-Min Sketch	CUSUM	Detection Rate, False Alarm Rate	Network Traces
	Eswaran et al. [71]	SPOTLIGHT	Similarity Threshold	Precision, Recall	DARPA, Enron, New York Taxi
	Cao et al. [71]	Linear Projection Sketch	GLR Test	ARL, EDD	
Community Analysis	Asur et al. [10]	Markov Cluster	Pattern Matching	Custom Metrics	DBLP
	Duan et al. [60]	Compactness Merge	Similarity Threshold	Precision, Recall	Enron
	Chen et al. [59]	Community Representative	Pattern Matching	Custom Metrics, Speedup	Synthetic dataset
Compression Cost Analysis	Cheng et al. [5]	Graph Signal Processing	Similarity Threshold	Detection Accuracy	Enron
	Chakrabarti et al. [61]	MDL	Similarity Threshold	Custom Metrics	Epinions, DBLP
	Sun et al. [62]	MDL	Similarity Threshold	Compression Gain, Scalability	Enron
	Xu et al. [63]	MDL	Clustering		
	Xu et al. [3]	MDL	Clustering		
Graph Kernels	Gao et al. [64]	Graphlets	Classification	Precision, Recall, F_1	Enron, MIT
	Zhang et al. [40]	Weisfeiler-Lehman Kernel	One-class SVM	Precision, Recall	AMGP
	Zambon et al. [82]	Graph prototypes	CUSUM	False Alarm Rate, Precision	MUTAG, ENZYMES
	Zambon et al. [65]	Attribute kernel	CUSUM	ARL	Synthetic dataset
Reconstruction Residual Analysis	Li et al. [66]	Linear Reconstruction	Hypothesis Test	Recall, FAR, RTE, ARI	IEEG Seizures
	Peng et al. [67]	CUR Decomposition	Residual Threshold	AUC	Amazon Co-purchase, Enron
			Residual Threshold	AUC	Amazon Co-purchase, Enron

representation techniques and change point detection methods can be used to capture topological features. Methods, evaluation metrics, and datasets are summarized in Table III and will be further detailed in Section VI.

1) *Attribute Analysis*: A basic method for describing graph topology is to use simple attributes, which can help identify change points in graph snapshot time series. In [45], several graph features are introduced:

- 1) *Size*: Number of edges in a graph.
- 2) *Maximum node degree*: Maximum edges connected to a single node.
- 3) *Scan statistic*: Computed as

$$s(\mathbf{X}) = \max \{l_{w_i}(\mathbf{X})\},$$

where l_{w_i} is the local statistic for the i th sliding window. In graphs, the scan statistic for the k -neighborhood of a vertex is defined as

$$s(G) = \max \{l_{N_k(v)}(G)\}.$$

In [45], change points are detected when attribute observations deviate from the average. Since a single feature may not fully capture graph topology, the authors propose fusing features using the statistic $S^p(t) = \sum_{i=1}^n p_i(t)S_i(t)$, where p_i are fusion weights calculated via Monte Carlo methods. Reference [46] refines these weights, and [47] introduces scan statistics related to community structures and temporal normalization.

In [36], a multi-view analysis combines global, sub-graph, and node-level attributes into a single fusion attribute $f = \alpha f_n + \beta f_c + \gamma f_g$, with α , β , and γ as the respective coefficients. Changes are detected if the divergence between regressed and real-world results exceeds a threshold.

In [84], graph snapshots are clustered based on node importance scores, and changes are detected when a snapshot forms a new cluster.

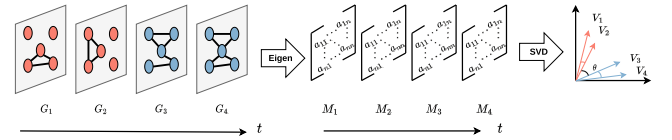


Fig. 5. Change point detection in graph snapshots uses eigenvectors and Singular Value Decomposition (SVD). Matrices M_1 , M_2 , M_3 , and M_4 are the eigenmatrices of snapshots G_1 , G_2 , G_3 , and G_4 . After SVD, the eigenvectors are V_1 , V_2 , V_3 , and V_4 . A significant change occurs between G_2 and G_3 .

Graph attributes are computationally efficient and scalable. For example, the time complexity to compute scan statistics in a graph is $O(n \times n_{neighbor}(V))$, where n is the number of nodes and $n_{neighbor}(V)$ is the selected neighbor size [127]. These methods are effective and can be easily extended to large graphs. However, this approach may not always accurately pinpoint all change points over time.

2) *Matrix Factorization*: Matrix factorization extracts features from a graph's adjacency matrix, assuming a constant number of nodes. Changes are detected by monitoring vector distances, such as cosine similarity and euclidean distance, as shown in Fig. 5.

One approach directly factorizes adjacency matrices. In [48], for SMS network changes, features on N customers are observed over time using a sliding window of length W . An $N \times N$ adjacency matrix is created for each time window, and changes are detected by comparing the "eigen-behavior vector" $\mathbf{u}(t)$ at time t with the average vector $\mathbf{r}(t-1)$ from previous windows. Reference [49] calculates eigenvector divergences between consecutive snapshots using energy distance, analyzed with an EWMA control chart. Reference [71] maps high-dimensional data to a low-dimensional space using a projection matrix and detects changes with CUSUM.

Laplacian matrix-based techniques are also explored. In [11], a "context matrix" is constructed from Laplacian eigenvectors,

and changes are detected by comparing Laplacian spectrum vectors using cosine similarity. Reference [50] applies Fourier transform to the Laplacian matrix, fits a Gaussian model to eigenvectors, and uses CUSUM to monitor changes. Reference [51] introduces a Spectral Scan Statistic (SSS) on the graph Laplacian. Change points are detected using the generalized likelihood ratio test (GLRT). Experiments demonstrate the method's superiority over naive detectors such as edge thresholding and χ^2 testing.

Graph operator-based techniques are discussed in [4], where a Bethe-Hessian matrix represents community relations, and changes are detected using CUSUM measures.

Matrix factorization methods are effective for detecting topological changes but have high time and memory costs. For example, SVD on an adjacency matrix has a time complexity of $O(n^3)$ [111], resulting in $O(n^3T)$ for a time period T , which can be impractical for large graphs.

3) *Probabilistic Methods*: Probabilistic methods simulate graph snapshot evolution using stochastic models to detect change points based on changes in fitting parameters. Common methods include Markov chains [52] and stochastic graph models [53].

Markov chains model event sequences where each event depends only on the previous state. In [52], a Markov chain tracks k edges with a sliding window w over the graph time series. The edge state transitions (forming, vanishing, etc.) are linked with probabilities, and changes are detected by shifts in probability between windows.

Stochastic graph models, such as the Erdos-Renyi [128], stochastic block [129], latent position [130], and random dot product models [131], model node associations using probability distributions. The stochastic block model groups nodes into K clusters with high intra-group connectivity. The latent position model assigns each node a latent vector, with connectivity probabilities modeled by a function $f(\mathbf{z}_i, \mathbf{z}_j)$. The random dot product model uses the inner product $\langle \mathbf{z}_i, \mathbf{z}_j \rangle$ for connectivity.

Fitting stochastic models to real-world data is often done with methods like maximum likelihood estimation (MLE) or Bayesian inference. In [53], Bayesian inference is used with a hierarchical random graph model, while [54] applies MLE to a stochastic block model, monitored by an EWMA control chart. Reference [55] and [56] use Adjacency Spectral Embedding with the random dot product model, and detecting changes using CUSUM.

Additionally, [132] and [133] model adjacency matrix entries as Bernoulli random variables Ω , detecting changes with CUSUM when observations of Ω change significantly.

Probabilistic methods detect changes using fitted model parameters from Markov processes or random graph models, with techniques like MLE, which scales to large graphs with time complexity $O(nT)$, where n is the number of nodes and T is the time length [134].

4) *Sampling-Based Methods*: Sampling-based approaches are used to manage large graphs by focusing on a subset of nodes and edges to reduce computational costs. Common methods include random walk and sketch techniques, which capture local graph topology to detect change points by observing significant shifts in sampled features.

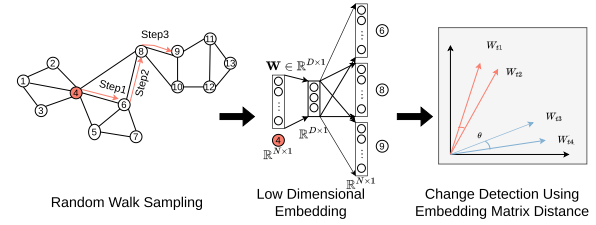


Fig. 6. Change point detection in graph snapshots uses random walk analysis. Node embedding is computed with the skip-gram model [135]. Detection monitors the distance between consecutive graph embeddings.

One approach uses short random walks [135] to capture local views of graphs. For a graph with N nodes, each node i is represented by a one-hot vector $\mathbf{z}_i \in \mathbb{R}^{N \times 1}$. The goal is to learn an embedding matrix $\mathbf{W} \in \mathbb{R}^{D \times N}$ mapping $\mathbf{z}_i$ to a D -dimensional space. A l -step random walk from node i records a neighborhood set $\mathcal{N}_i$. The embedding matrix $\mathbf{W}$ is optimized to maximize the probability of neighbors in $\mathcal{N}_i$ using Maximum Likelihood Estimation (MLE). Details are illustrated in Fig. 6. Reference [57] extends this with an Autoencoder for random walk embedding, using the Hadamard product for edge vectors, which are clustered to detect changes when new edges deviate from existing clusters.

Graph sketching methods select parts of the graph to create local views, combining these for a global view. Reference [81] uses real-time streaming to construct a Count-Min Sketch [58], updating a hash table with source and destination IPs and detecting changes using CUSUM. Reference [136] introduces SPOTLIGHT, which sketches node degrees to detect changes in connection density between snapshots. Sketches are projected to a low-dimensional space, where significant changes in vector distances indicate graph changes.

Sampling-based methods are efficient for large graphs. Random walk methods have a time complexity of $O(n)$ [57], with dynamic graph change detection at $O(nT)$, which is manageable for large graphs. Graph sketches also provide efficient representations. Count-Min sketches have a time complexity of $O(nd)$ [58], where n is the number of graph nodes and d is the number of hash functions. However, their performance may degrade with small graphs due to repetitive sampling.

5) *Community Analysis*: Community analysis assumes that network interactions are modular, allowing nodes to be grouped into communities [10]. Detecting changes in community structures can reveal topological changes in dynamic graphs.

In [10], five community events are defined: continue, k -merge, k -split, form, and dissolve. Communities are detected using the Markov Cluster (MCL) Algorithm [137], creating a $K_i \times N$ binary matrix $\mathbf{T}_i$ where K_i is the number of communities and N is the number of nodes. $\mathbf{T}_i[k][j]$ is 1 if node j belongs to community k . Events between timestamps are computed via bitwise operations on $\mathbf{T}_i$ and $\mathbf{T}_{i+1}$. Reference [59] also predefines community events but uses representative nodes for change detection instead of enumerating all communities.

In [5], communities are represented as subgraphs with adjacency matrices. Spectral methods are applied to stacked matrices to obtain a low-dimensional representation. A large matrix is formed from diagonal stacking of community matrices,

smoothed using Fourier Transform and filtering, and change detection is performed on the spectral signal.

In [60], a two-step approach is used: first, compact communities are constructed based on node compactness [138], then these are merged to maximize modularity. Change points are detected by measuring partition similarity over time, with an incremental algorithm, Inc-Group, updating partitions as new graphs arrive.

Community-based change point detection methods are computationally intensive, often involving algorithms like MCL [137] or graph signal processing, both with time complexity $O(n^3)$. For T time snapshots, the complexity is $O(n^3T)$, making these methods impractical for large graphs.

6) *Compression Cost Analysis*: Compression cost analysis follows the Minimum Description Length (MDL) principle [139], which suggests that patterns in graph snapshots can be compactly represented using methods like Huffman coding. A significant change in compression cost indicates a change in the graph.

In [61], nodes in a graph snapshot are grouped into K clusters, and the adjacency matrix is rearranged to form homogeneous blocks. Changes are detected by monitoring deviations in encoding cost. Reference [62] focuses on community structure, where nodes with communication links form communities, and the encoding cost reflects both the graph and its community structure. Reference [63] integrates node clustering into the MDL-based compression, while [3] uses EM-clustering for community modeling and a bottom-up search to find changes by comparing encoding costs of merged snapshots.

Compression cost analysis methods are based on the Minimum Description Length (MDL) principle. The computational cost depends on the choice of description objectives to be minimized and the method used to describe the entire graph. In most cases, however, the computational cost arises from two main aspects: determining the optimal number of clusters k and performing iterations to group graph nodes into clusters to achieve the minimum description length. For example, in [61], the time complexity is $O(\omega(D)(k^*)^2I)$, where k^* represents the optimal number of clusters, I denotes the number of iterations to group nodes, and $\omega(D)$ is the cost of iterating through the adjacency matrix to execute the grouping algorithm. Considering that the graph snapshot series has a length of T , the computational cost renders these methods impractical for large graphs.

7) *Graph Kernels*: In graph analysis, common kernels include graphlets (subgraph-based) and Weisfeiler-Lehman (neighborhood aggregation). Changes in graph kernels indicate changes in graph snapshots.

The graphlets kernel represents a graph by counting occurrences of small, non-isomorphic subgraphs. These counts form a vector that serves as the graph's "signature" (Fig. 7). Reference [64] uses graphlet-based signatures for malware detection by comparing function call graphs of Android programs.

The Weisfeiler-Lehman kernel refines node colors iteratively using:

$$c^{(k+1)}(v) = \text{HASH} \left(\left\{ c^{(k)}(v), \left\{ c^{(k)}(u) \right\}_{u \in \mathcal{N}_v} \right\} \right)$$

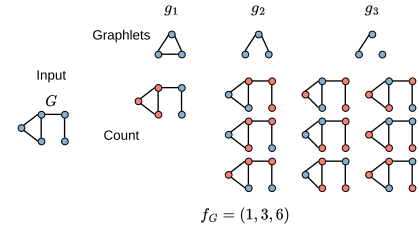


Fig. 7. An example of constructing graphlets vectors by counting graphlets [140].

where $\mathcal{N}_v$ are the neighbors of v . It stops when node colors stabilize, and a color vector represents the graph's topology. Reference [40] uses this kernel for anomaly detection in graph data.

Reference [65] formalizes graphs as objects in a graph domain belonging to the family of graph alignment spaces (GAS) by using attribute kernels [141]. This approach transforms graphs into vectors in euclidean space, allowing change point detection to be applied to these transformed vectors. The authors derive the test statistics within the GAS framework and use hypothesis testing to identify changes in the series of transformed vectors.

Reference [82] embeds graph snapshots into vectors using graph prototypes. For a given graph G , the best matching prototype R is found by comparing the Graph Edit Distance (GED) [142] between the selected graph snapshot and a set of predefined prototypes $\mathcal{R} = \{R_1, R_2, \dots, R_m\}$. Changes are detected using the CUSUM algorithm on the extracted prototypes.

Extracting graph kernels is computationally intensive, particularly for large graphs. In general, counting graphlets of size k (where k is the number of vertices in the graphlet) is NP-hard due to the need to check all possible subsets of vertices to determine if they form the desired graphlet. When analyzing the time complexity of the Weisfeiler-Lehman kernel, [143] presents a polynomial-time algorithm with complexity $O_k(n^{O(k)})$ for graphs with a color multiplicity of at most 5. This algorithm achieves k -WL identification by reducing the problem to the separability of 2-induced coherent configurations. Here, k denotes the dimensionality of the substructures (or subgraphs) considered by the kernel when comparing graphs, specifically referring to the vertex subsets used in the algorithm.

8) *Reconstruction Residual Analysis*: Reconstruction residual analysis uses the reconstruction residuals which is the difference between the estimated data generated by the approximation model and real-world observations to detect changes. For an attributed graph at time t , the estimated data $\hat{\mathbf{X}}$ is based on principal components. The reconstruction residual R , defined as $R = \|\mathbf{X} - \hat{\mathbf{X}}\|_2^2$, measures the deviation between true data $\mathbf{X}$ and its estimation. Significant fluctuations in R indicate changes in the time series.

Reference [66] introduces *radar*, a residual-based tool for attributed graphs with N nodes and D features. The attribute matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ and adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ are used to estimate $\mathbf{X}$ as $\hat{\mathbf{X}} = \mathbf{W}\mathbf{X}$, minimizing: $\|\mathbf{X} - \mathbf{W}\mathbf{X}\|_F^2 + \alpha\|\mathbf{W}\|_{2,1}$. Incorporating residuals, the objective becomes: $\|\mathbf{X} - \mathbf{W}\mathbf{X} - \mathbf{R}\|_F^2 + \alpha\|\mathbf{W}\|_{2,1} + \beta\|\mathbf{R}\|_{2,1} + \gamma\text{tr}(\mathbf{R}^T\mathbf{L}\mathbf{R})$ where $\mathbf{R}$ is the residual matrix, $\mathbf{D}$ is the degree

TABLE IV
SUMMARY OF SURVEYED WORKS FOR CHANGE POINT DETECTION IN TIME SERIES ON GRAPHS

Method Class	Reference	Dynamic Graph Model	Change Detection Method	Evaluation Metrics	Application
Direct Detection	Tartakovsky et al. [68]	None	CUSUM	ADD, FAR	Synthetic dataset on TCP and UDP network
	Liu et al. [37]	None	CUSUM	ARL, EDD	Synthetic dataset on communication network
	Hundman et al. [43]	None	LSTM Prediction	Precision, Recall, $F_{0.5}$	SMAF, MSL
Dimension Reduction	Lakhina et al. [69]	PCA	EWMA on Residual	Recall, FAR	Sampled network flow data
	Pajouh et al. [70]	LDA	Classification	Recall, FAR	NSL-KDD
Correlation Analysis	Tan et al. [29]	Correlation	Anomaly Threshold	ROC	KDD-99
	Tan et al. [72]	Correlation	Anomaly Threshold	ROC	KDD-99
	Tan et al. [73]	Correlation	Anomaly Threshold	Precision, FAR	KDD-99
	Barnett et al. [74]	Correlation	Bootstrap Procedure	Statistical Power	Stock market, fMRI
Entropy Analysis	Lakhina et al. [75]	PCA, Entropy	Clustering	Recall, FAR	Sampled network flow data
	Berezinski et al. [76]	Entropy	Threshold	Recall, FAR	Sampled network flow data
	Petkovic et al. [77]	Entropy	Threshold	Recall, FAR	Synthetic dataset
Deep Learning	Canizo et al. [32]	CNN convolution	RNN Classification	Precision, Recall	Sampled Elevator Sensor data
	Erfani et al. [39]	RBM	One-class SVM	AUC	Gas, Human Activity Recognition
	Garg et al. [78]	RBM	DBN	Precision, Recall, FAR, F_1 score	KDD-99
	Su et al. [79]	VAE	Reconstruction Residual Threshold	Precision, Recall, F_1	SMD, SMAF, MSL
	Audibert et al. [80]	AE with GAN	Reconstruction Residual Threshold	Precision, Recall, F_1	SMD, SMAF, SWaT, WADI
	Zhao et al. [42]	GAT	Prediction and Reconstruction Residual Threshold	Precision, Recall, F_1	SMAF, MSL
	Li et al. [44]	GAN	Constructor and Discriminator loss	Precision, Recall, F_1	SWaT, WADI, KDD-99

matrix, $\mathbf{L}$ is the Laplacian matrix, and α , β , and γ are coefficients. Timestamps with the largest k residuals are flagged as changes. Reference [67] further refines this by using CUR decomposition [144] to reduce noise and irrelevant attributes.

The computational cost of residual analysis often arises from fitting the parameters of the approximation model to describe the topological structures of the original inputs. For example, in the *radar* framework [66], the objective function is minimized by directly solving the equations to set the derivatives of the hyper-parameter to zero, resulting in a time complexity of $O(N^2)$, where N is the number of graph nodes. Consequently, reconstruction residual analysis is not well-suited for detecting changes in large dynamic graphs.

B. Detection in Time Series on Graphs

Time series on graphs involve multiple time series with spatial and temporal features. Each entity's dynamics can impact its neighbors and the global graph attributes. For instance, traffic flows at intersections influence neighboring flows, creating congestion at other intersections. They may also vary with time, such as office areas being busier on weekdays and zoo areas on weekends.

Traditional time series methods (CUSUM, dimension reduction, entropy-based) can be adapted for graph time series, but may overlook graph structure, reducing accuracy. Advanced methods using spectral graph theory and graph neural networks offer improved performance. Summarized methods are listed in Table IV.

1) *Direct Detection*. *CUSUM*: The CUSUM algorithm [145], initially for univariate time series, can be adapted for time series on graphs by using matrices. It detects changes by comparing the likelihood ratio before and after a candidate change point [146]. In multivariate scenarios, $\mathbf{X}$ represents combined observations. Adaptations include creating a univariate observation from multiple data points, as seen in [68] where packet numbers are aggregated for CUSUM change detection. Alternatively, a local CUSUM statistic can be maintained in each observation dimension. Reference [37] proposes a consensus-based approach in distributed networks, where each sensor calculates a local CUSUM statistic and shares it with neighbors. In [118], sensors in an IoT network trigger consensus only when detecting significant changes locally.

Direct detection methods for time series on graphs have time complexity $O(T)$, where T is the time length. These methods are efficient with respect to time and memory but may struggle with deep changes due to their simplistic representations.

2) *Dimension Reduction*: Dimension reduction methods for high-dimensional time series on graphs identify principal vectors to either reconstruct data and monitor residuals for changes, or to use as features for classifiers.

In [69], PCA is used for network traffic analysis. Traffic data is reduced to principal components, separating into *normal* $\hat{S}$ (first 3-4 principal axes) and *abnormal* $\hat{S}$ (the remaining principal axes) subspaces. The data $\mathbf{X}$ is decomposed in the same way as $\mathbf{X} = \hat{\mathbf{X}} + \tilde{\mathbf{X}}$, where $\tilde{\mathbf{X}}$, the reconstruction residual, signals anomalies. Spikes on the residual indicating changes.

In [70], principal vectors are used as features for classifiers. PCA and LDA [147] project network traffic data into a subspace, with the resulting feature vectors used for change detection via classifiers like Naive Bayes or KNN.

Dimension reduction methods like PCA or LDA for change detection involve computing principal components and residuals. These methods, with time complexity $O(n^3)$ for correlation matrix factorization [69], may not be suitable for large graphs.

3) *Correlation Analysis*: Correlation analysis monitors relationships within multivariate time series, akin to detecting topological changes in graph snapshots. Since time series on graphs exhibit distinct spatial and temporal features, related time series often receive high scores in the correlation matrix. Therefore, this method acts as a powerful tool for detecting changes in the latent topological attributes of time series on graphs. Changes are identified when the global correlations between these series undergo alterations.

Multivariate Correlation Analysis (MCA), introduced in [29], detects Denial-of-Service (DoS) attacks by analyzing temporal and spatial features of network nodes. The method mines correlations between time sequences, with changes in multivariate correlations indicating state changes. When a DoS attack happens, the attacking and attacked nodes share distinct temporal features. These features can include the timing of attack packets, the duration of the attack, and the frequency of requests. Furthermore, if attackers conduct a distributed denial-of-service (DDoS) attack, the attacking nodes can share spatial features, such as originating from the same network segment or having similar IP address ranges. Identifying these spatial features can

aid in detecting and blocking malicious traffic. Therefore, the spatial and temporal correlations between these nodes can be high. Correlations are represented using euclidean Distance Maps (EDM), and deviations are monitored using Mahalanobis Distance. An alert is triggered if Mahalanobis Distances exceed a threshold. Reference [73] replaces Mahalanobis Distance with Earth Mover's Distance for better performance. Reference [74] outlines a comprehensive approach for detecting change points in correlation networks, focusing on the calculation of the correlation matrix and the use of distance metrics. The method begins by assuming a system is composed of a fixed number of nodes, with characteristics observed at distinct time points. These characteristics are represented as a matrix where each column is an n -dimensional vector of observations. The method standardizes these observations by centering them to have a temporal mean of 0 and scaling them to have unit variance. To detect change points, the method defines a set of diagonal matrices that are used to calculate the sample covariance matrix over subsets of the data ranging from one column to another, corresponding to specific time points. The covariance matrix is computed as the sum of the product of the data vectors, normalized by the number of observations in the subset.

Correlation-based methods require continuous monitoring and iterative calculations of correlation matrices. The time complexity is $O(mn^2)$ [72], where n is the number of variables and m is the number of observations. This method becomes computationally expensive for large dynamic graphs, making it less suitable for CPD in large graphs.

4) *Entropy Analysis*: Entropy analysis, like MCA, detects Denial-of-Service (DoS) attacks by examining randomness in packet address or port distributions [75], [76], [77]. Common entropy types include Shannon entropy [148], Renyi entropy, and Tsallis entropy [149]. Shannon entropy, which balances contributions from distribution mass and tail, is the most frequently used. Renyi and Tsallis entropy generalizations offer parameterized versions of Shannon entropy.

For a network with m nodes, let $\mathbf{X} = \{x_1, x_2, \dots, x_n\}$ be all captured packets at time t , and $\mathbf{C} = \{c_1, c_2, \dots, c_m\}$ be the destination packet counts. Shannon entropy of packet destinations is calculated as: $H_s = -\sum_{i=1}^m \frac{c_i}{|\mathbf{X}|} \log_a \frac{c_i}{|\mathbf{X}|}$. Changes in entropy can signal network modifications. For example, a port scan attack can cause a decrease in entropy of destination IPs and an increase in destination ports.

Entropy-based methods for graph change detection calculate global and group-specific observation counts with a single graph traversal, resulting in a time complexity of $O(n)$, where n is the number of nodes. These methods remain effective even as graph size grows.

5) *Deep Learning*: Deep-learning-based approaches leverage deep neural networks to minimize feature engineering efforts, extracting spatial and temporal features in time series on graphs end-to-end. In this section, we first introduce some basic deep learning blocks and their design philosophies for capturing temporal and spatial features in time series on graphs. Then, we introduce commonly adopted frameworks for detecting changes in time series on graphs.

CNN: CNN blocks in temporal tasks use convolution and pooling operations hierarchically to capture long-range temporal patterns [150], [151]. In real-world practice, CNN blocks are often organized in encoder-decoder structures [152] or incorporated with GNN building blocks [151] to capture features in dynamic graphs.

RNN: RNN blocks, including GRU and LSTM [43], capture temporal features by iterating the model through temporal observations while maintaining a hidden state that describes the features of the temporal series [153]. The hidden state completes the encoding process once all the input observations in a batch have been iterated. RNN blocks are widely used for change detection using prediction-based methods [43].

Transformer: Transformers are highly effective deep learning building blocks that utilize the attention mechanism [154], [155], [156]. Each observation in the input sequence is assigned attention scores denoting how much weight the model should assign to in the temporal context. The hyperparameter matrix is updated using a backpropagation algorithm within the CPD framework.

GNN: GNN blocks, including GCN [155], GraphSAGE [157], and GAT [158], are common components in deep learning models for capturing spatial features in graphs [159]. Given a node and its related attributes in a graph, GNN blocks first compute messages based on the node's attributes and its neighbors' attributes, and then aggregate these messages to form a forward loop. Spatial features are encoded in the iterative forward loop [90].

In the following paragraphs, we will introduce commonly adopted frameworks.

Prediction-based framework: This type of framework detects change points in time series mainly in two ways. One approach uses the deviation between predicted values and real-world observations to detect changes [43]. The other approach classifies series of observations into changes and non-changes. In [32], a multi-head CNN-RNN framework detects changes in industrial scenarios. Sensor data series are organized in a high-dimensional matrix. A CNN applies 1-D temporal convolution to extract features, which are then stacked and sent to an RNN classifier to identify change points.

AE: Autoencoders map high-dimensional input data into a low-dimensional latent space and reconstruct the input. The encoder maps data to the latent space: $\mathbf{L} = E(\mathbf{X})$, and the decoder reconstructs the input: $\hat{\mathbf{X}} = D(\mathbf{L}) = D(E(\mathbf{X}))$, aiming for $\hat{\mathbf{X}}$ to be identical to $\mathbf{X}$. Changes can be detected by monitoring reconstruction residuals or using the latent representation for classification, as shown in Fig. 8. In [39] and [78], RBM and DBN are employed for dimension reduction in multivariate time series data, and feature vectors are sent to an SVM classifier for change detection. In [160], the authors use Adversarial Autoencoders (AAEs) for graph embedding on Constant Curvature Manifolds (CCMs) to detect changes in graph streams. The method uses Graph Convolutional Layers to learn the graph's structure, followed by a Global Pooling Layer to aggregate node-level embeddings. Two approaches ensure embeddings adhere to CCMs: matching the

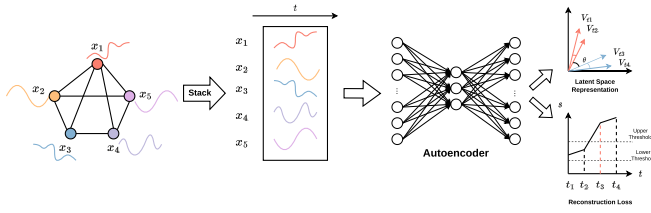


Fig. 8. Change point detection in graph time series using an Autoencoder. The Autoencoder provides a latent space representation or reconstruction loss, both serving as indicators for change detection.

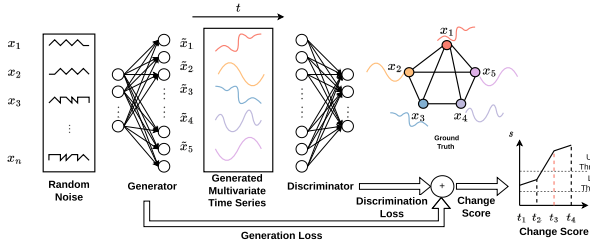


Fig. 9. Change point detection in graph time series using a GAN-based approach. The generator produces construction loss from random noise, and the discriminator calculates discrimination loss between constructed and ground truth data. These losses, combined with the CUSUM/EWMA control chart, detect changes.

prior distribution in latent space and using a parameter-free geometric discriminator. In [79], a Variational Autoencoder (VAE) learns latent space representations from input time series data, using reconstruction loss for change point detection. Sometimes, AEs use combinations of deep learning blocks and architectures for improved encoding performance. In [80], AEs are combined with GAN to enhance training. In [42], MTAD-GAT enhances anomaly detection in multivariate time-series data using two parallel graph attention (GAT) layers. The feature-oriented GAT layer captures causal relationships between features, while the time-oriented GAT layer models temporal dependencies within the data. This dual-layer approach, combined with a Gated Recurrent Unit (GRU) for sequential pattern recognition and joint optimization of forecasting and reconstruction models, creates a robust anomaly detection system.

GAN: GAN is a generator-discriminator framework where the generator creates fake data, and the discriminator distinguishes it from real data. The training is a min-max game, resulting in the generator mapping latent space to real data and the discriminator capturing real data's probability distribution. Both can detect anomalies in time sequences by monitoring reconstruction residuals from the latent space (illustrated in Fig. 9). In [44], an LSTM-GAN framework uses reconstruction loss and discriminator predictions to generate an anomaly score.

Deep learning blocks help uncover latent temporal and spatial features among entities in dynamic graphs [28], [90]. They offer a promising direction for detecting change points in dynamic graphs. In [161] and [162], the authors clearly describe the overall CPD pipeline using an autoencoder-based framework to illustrate that the model should first be trained on predefined normal data or batches sampled during warm-up phases to capture frequent (or “normal”) time series patterns before changes

are detected. The computational cost becomes prohibitive when the training time is too long for large-scale datasets.

VI. DATASETS AND EVALUATION METRICS

Effective design and implementation of a change point detection framework require appropriate datasets for testing purposes and evaluation metrics. This section lists publicly available datasets across various research domains and related evaluation metrics from the literature.

A. Public Datasets

Tables V and VI summarize commonly used datasets for CPD in both graph snapshot series and time series on graphs, categorized by application fields. In Table V, the scale of each dataset is provided, where Num_N and Num_E represent the estimated numbers of nodes and edges in a dynamic graph. In Table VI, since the connected time series on graphs evolve over time, the size of the graph is not given. Instead, we provide the number of records in these datasets in the column “duration”. For graph snapshot series, widely researched datasets include DBLP [163] and ACM [164] (author collaboration networks), Enron [53] (internal email communication records at Enron), and MIT [53], [84], [112] (student contact data at MIT). For time series on graphs, commonly researched datasets include the New York Taxi dataset [165] (taxi trips in New York City), KDD-99 [29] (intrusion detection), and SWaT [166], [80] and WADI [80] (industrial water treatment network flow).

B. Evaluation Metrics for Offline Training

For most change point detection frameworks based on supervised and semi-supervised learning, evaluation metrics are mostly based on the confusion matrix shown in Table VII. Some commonly used metrics are listed below.

- **Precision:** The ratio of correctly detected change points to all predicted change points, indicating the accuracy of the framework, calculated as
$$P = \frac{TP}{TP + FP}.$$
- **Recall:** The ratio of correctly detected change points to all actual change points, indicating the sensibility of the framework, calculated as
$$R = \frac{TP}{TP + FN}.$$
 Alternatives include *hits at n* ($H@n$) [11], which reports the number of identified significant anomalies out of the top n most anomalous points, and *absolute error* [55] between the estimated and actual number of change points.
- **F1-score:** The harmonic mean of precision and recall, calculated as
$$F_1 = \frac{2PR}{P + R}.$$
- **ROC-curve and AUC:** ROC plots *true positive rate* (TPR) against *false positive rate* (FPR), with FPR computed as
$$FPR = \frac{FP}{FP + TN}.$$
 AUC is the area under the ROC curve; higher AUC indicates better performance.

TABLE V
COLLECTION OF PUBLIC DATASETS WITH NUMBER OF NODES AND EDGES, DESCRIPTION, AND URL FOR CPD IN GRAPH SNAPSHOT SERIES

Category	Dataset	Num. N	Num. E	Description	URL
Citation network	DBLP [163]	5.3×10^6	3.7×10^7	Citation network of DBLP	https://originalfileserv.aminer.cn/misc/dblp_v14.tar.gz
	ACM [164]	2.3×10^6	9.7×10^6	Citation network of ACM	https://ls.aminer.cn/lab-datasets/citation/acm.v9.zip
	Cora [167]	2.3×10^4	9.2×10^4	Cora citation network	http://konect.cc/files/download.tsvsubelj_cora.tar.b2
	Arxiv [168]	1.35×10^6	6.7×10^6	ArXiv citation network	https://github.com/mattbierbaum/arxiv-public-datasets
	Citeseer [169]	3.3×10^3	4.7×10^3	Citation network of Citeseer	https://lincs.org/datasets/#citeseer-doc-classification
Communication network	Eron [53]	8.7×10^4	1.5×10^6	Enron email network	https://www.cs.cmu.edu/~enron/
	Wikipedia Talk [170]	2.4×10^6	5.0×10^6	Communication network of Wikipedia	https://snap.stanford.edu/data/wiki-Talk.html
	DNC Email [171]	2×10^3	1.2×10^5	Democratic National Committee email network	https://networks.skewed.de/net/dnc
	Linux Kernel Reply [172]	6.3×10^4	1.1×10^6	Linux kernel mailing network	http://konect.cc/networks/lkml-reply/
	Digg [173], [174]	7.7×10^5	5.9×10^6	Digg reply network	https://networkrepository.com/soc-digg.php
Social network	UCI Message [11]	1.9×10^3	6.0×10^4	Online community of students from the University of California, Irvine	http://konect.cc/networks/opsahl-ucsocial
	Twitter [174], [175], [176]	4.1×10^7	1.5×10^9	Follower network from Twitter	https://anlab-kaist.github.io/traces/WWW2010
	Google+ [177], [178]	2.8×10^7	4.6×10^8	Google+ social structure	https://gonglab.pratt.duke.edu/google-dataset
	YouTube [179]	3.2×10^7	9.4×10^7	Social network of YouTube Users	https://socialnetworks.mpi-sws.org/data-wosn2008.html
	Flickr [180]	2.3×10^6	3.3×10^7	Social network of Flickr	https://socialnetworks.mpi-sws.org/data-wosn2008.html
Computer network	Epinions [10]	7.6×10^4	5.0×10^5	Who-trust-whom online social network	https://snap.stanford.edu/data/soc-Epinions1.html
	CAIDA [181]	2.6×10^4	5.3×10^4	CAIDA autonomous system network	https://snap.stanford.edu/data/as-aida.html
	Skitter [168]	1.7×10^6	1.1×10^7	Skitter autonomous system network	https://snap.stanford.edu/data/as-skitter.html
	Gnutella [182]	6.2×10^4	1.5×10^5	Network of Gnutella hosts	https://snap.stanford.edu/data/p2p-Gnutella31.html
	Internet topology [183]	3.5×10^4	1.7×10^5	Connections between autonomous systems	http://konect.cc/networks/topology/
Neural network	Caenorhabditis elegans (neural) [184]	3.0×10^2	4.3×10^3	Neural network in brain	https://sites.cc.gatech.edu/dimacs10/archive/clustering.shtml
Economy	World Trade [36]	196	-	Trade flows of 196 countries from 1948 to 2000	http://ksgleditsch.com/exptrade/gdp.html
	Amazon Co-purchasing [66]	3.3×10^5	9.3×10^5	Co-purchased product in Amazon	https://snap.stanford.edu/data/com-Amazon.html
Reality Mining	MIT [53], [84], [112]	96	1.0×10^6	human contact data among 100 students of MIT	https://www.cs.cornell.edu/~arb/data/congress-bills/
	Legislative Cosponsorship Network [84]	1718	4.2×10^5	Co-sponsors of legislative bills in US Congress	https://data.cincinnati-oh.gov/safety/
	PDI [63]	31	77	Crime incidents in Cincinnati	https://pdi-police-data-initiative-crime-incidents/k59e-2pvt/about_data
Smart City	Western States Power Grid [71]	4941	6594	Western States Power Grid of the United States	https://networkdata.ics.uci.edu/data/power/

TABLE VI
COLLECTION OF PUBLIC DATASETS WITH DURATION, DESCRIPTION, AND URL FOR CPD IN TIME SERIES ON GRAPHS

Category	Dataset	Duration	Description	URL
Traffic network	New York Taxi [165]	2009-2018	Taxi trip records in New York	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
	New York City Bike [185]	2015-2017	Bike trip records in New York	https://citibikenyc.com/system-data
	Divvy Bikes [186]	2013-2024	Divvy bike trip data	https://divvybikes.com/system-data
	PeMS [187]	2001-2013	Data collected from California free way system	https://dot.ca.gov/programs/traffic-operations/mptr/pems-source
Weather and climate	US weather outliers [188]	1967-now	Historical U.S. temperature outliers	https://data.world/carlweis/us-weather-outliers-1964
	ACRON-SAT [189]	1910-now	Australian weather data	http://www.bom.gov.au/climate/data/acorn-sat/#tabs-data-and-networks
	KNMI [190], [191]	1961-now	Historical weather data in Netherlands	https://dataplatform.knmi.nl/
	Climate [192]	1930-now	Global climate data	https://en.tutiempo.net/climate
Health care	California Temperature [63]	1960-now	Historical temperature in California	https://www.prism.oregonstate.edu/explorer/map.php
	Chickensox cases [193]	2005-2015	Weekly chickensox cases from Hungary	https://archive.ics.uci.edu/dataset/580/hungarian+chickensox+cases
	US cancer rates [194]	1999-2013	Cancer rate across U.S.	https://data.world/adamhelsinger/cancer-rates-by-us-state
	Ebola cases [195], [196]	2014-2016	Ebola data in record	https://data.world/brianray/ebola-cases
People migration	Covid-19 [197], [198]	2019-2024	Covid-19 data in Mexico	https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7865094/
	iEEG Seizures [65]	3.3×10^4 records	Detect seizure in intracranial EEG recordings	https://www.kaggle.com/c/seizure-detection/data
	Ground swell [199]	2000-now	Cross-border migration data	https://www.earthdata.nasa.gov/learn/articles/sedac-groundswell-dataset
	International Migration [200]	2000-2022	International migration data	https://stats.oecd.org/Index.aspx?DataSetCode=MIG
Cyber security	SENSORNETGUARD [201]	2023.01-2023.06	Dataset for identifying malicious sensor nodes	https://ieee-dataport.org/documents/sensornetguard-dataset-identifying-malicious-sensor-nodes
	WSN-DS [202], [203]	-	Dataset for intrusion detection	https://sel.psu.edu.sa/Research/datasets/2016_WSN-DS.php
	DARPA [136]	5 weeks in 1999	Intrusion detection dataset	https://www.ll.mit.edu/r-d/datasets/1999-darpa-intrusion-detection-evaluation-dataset
	KDD-99 [29]	4.9×10^6 records	Network intrusion detection dataset	https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html
Industry	NSL-KDD [70]	4.9×10^6 records	New version of the KDD99 intrusion detection dataset	https://www.unb.ca/cic/datasets/nsll.html
	SMD [79], [80]	5 weeks	Anomaly detection dataset collected from a Internet Company	https://github.com/NetManAIops/OmniAnomaly
	GAS [39]	1.4×10^4 records	Gas delivery sensor data	https://archive.ics.uci.edu/dataset/224/gas+sensor+array+drift+dataset
	SMAP [43], [79]	4.3×10^5 records	Sensor data from Soil Moisture Active Passive satellite	https://github.com/khundman/telemanom
Human activity	MSL [43], [79]	6.7×10^4 records	Sensor data from Curiosity Rover on Mars	https://github.com/khundman/telemanom
	SWaT [80], [166]	11 days	Network flow data in an industrial water treatment plant	https://trust.sutd.edu.sg/trust-labs_datasets/dataset_info
	WADI [80]	16 days	Water distribution data of an industrial water treatment plant	https://trust.sutd.edu.sg/trust-labs_datasets/dataset_info
	OPPORTUNITY Activity Recognition [39]	2.5×10^3 records	Human Activity Recognition dataset	https://archive.ics.uci.edu/dataset/226/opportunity+activity+recognition
	Daily and Sports(DAS) [39]	9.1×10^3 records	Sports dataset	https://archive.ics.uci.edu/dataset/256/daily-and-sports+activities
	Human Activity Recognition [39]	1.0×10^4 records	Human Activity Recognition database	https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones

TABLE VII
CONFUSION MATRIX FOR SUPERVISED AND SEMI-SUPERVISED CHANGE POINT DETECTION FRAMEWORK

	Predicted as change	Predicted as no-change
Actual change	TP	FN
Actual no-change	FP	TN

C. Evaluation Metrics for Online Detection

When implementing an online change point detection algorithm or deploying a framework trained offline for online use, real-time performance and accuracy are key metrics, estimated by *statistical power*, *average detection delay* (ADD), *average run length* (ARL), and *false alarm rate* (FAR).

- *Statistical power* [45], [46], [47], [51]. Measures the test's ability to detect an effect when there is one.
- *Average Detection Delay* [13], [38], [118]. The expected time difference between the reported change point t_d and the actual change point t_c : $ADD = E(t_d - t_c)$. An alternative is *relative time-step error* [65] between estimated and actual change point.
- *Average Run Length (ARL)* [49], [118], [204]. The expected duration before a false alarm: $ARL = E[t_f | t_r =$

$\infty]$, where t_f is the time when the first false alarm occurs, and t_r is the running time of the algorithm.

- *False Alarm Rate* [13], [38]. The rate of false reports when no actual change occurs: $FAR = \frac{1}{E(t_f)}$.
- *Adjusted Rand index (ARI)* [65]. Compares two partitions of the same set, with ARI ranging from -1 (disagreement) to 1 (perfect agreement).

D. Other Metrics

In some application-specific scenarios, scalability and resource consumption are also important factors for algorithmic design.

Scalability [50], [80], [89]: A change point detection algorithm based on matrix decomposition may work well on a small synthetic dataset but fail to run efficiently when there are thousands of nodes in graphs to monitor. Therefore, whether an algorithm still works well under scenarios with different amounts of inputs is a critical metric to measure the universality of a proposed algorithm.

Resource consumption [105], [118], [204]: Computations may require various types of resources, including CPU, GPU,

storage, and network. For example, if we want to deploy a designed change point detection algorithm into an IoT network, the communication and storage cost as well as the energy consumption are some of the most important factors that should be carefully considered. This metric varies for different computation scenarios and is often evaluated empirically.

VII. SUMMARY AND FUTURE DIRECTION

In this survey, we review change point detection techniques in dynamic graphs, discussing two forms of data and the challenges and applications of these techniques. We propose a new taxonomy for various detection techniques and suggest future research directions.

Effective graph representation: For better algorithm performance, an important issue for change point detection in dynamic graphs is to find an effective method to map high-dimensional data into a low-dimensional latent space to generate a representation that preserves the topology of graphs and attributes within nodes [87]. The representation techniques should consider not only the relationships between graph entities but also the correlations in the temporal dimension [7]. For some scenarios with limited computing capability, the embedding method should be computationally effective and energy-saving. For change point detection in large graphs, the graph representation techniques should also be more effective and scalable [86].

Online algorithm: Compared to an algorithm that should be trained offline and then deployed online for practice, pure online algorithms can be effective to achieve the quickest detection and able to detect changes sequentially without fine-tuning [38]. However, to perform effective online detection, the algorithm should be able to quickly recalculate graph representations after changes are detected in dynamic graphs. The online algorithm should also be flexible to fit the evolving dynamic graph when the distribution of collected data changes.

Detection on attributed and heterogeneous graphs: Most change point detection frameworks have been developed for homogeneous graphs. However, with the rapid prosperity of graph machine learning and multimodal learning [205], attributed and heterogeneous graphs have been used to represent relations between instances from different classes [66] [67], such as drug-effect networks [206]. Research on attributed and heterogeneous graphs helps reveal the relationships between different types of entities.

Location of changes in complex systems and explanation: After detecting state transitions, analyzing causality helps identify system vulnerabilities and evolution rules [36]. Recent works [14], [207] show potential for continuous learning and root cause analysis in dynamic systems.

Relationships between local and global changes: Investigating the relationships between local and global changes in dynamic graphs is crucial. Local changes, such as modifications in a small subgraph, can have significant global impacts on the network's structure and functionality [36]. For example, in a social network, an interest group can disband quickly if key

leaders leave, while the departure of less important members has minimal impact [59]. Similarly, in a traffic network, congestion at key intersections can affect a large area, while minor congestion on side roads may not cause significant disruption [208]. Developing methods to understand and model these relationships will enhance the ability to predict and manage broader network behavior. This involves leveraging algorithms that integrate local community detection with global structural analysis, ensuring that updates in local communities are efficiently reflected in the global network structure.

Discovering latent structures in dynamic graphs: While detecting changes in graph snapshots, the graph model can often be obtained through simple observations. However, hidden influences between graph entities may exist, which are not immediately apparent. Similarly, the spatial relationships in time series on graphs are not always clear, necessitating the development of effective data mining techniques to uncover such relationships [209]. Techniques such as dynamic graph neural networks [9], [210] and temporal embedding methods [211], [212] can help uncover these latent models. These approaches need to account for temporal dependencies and the evolution of node and edge attributes, enabling more accurate change detection.

Handling nodes joining or leaving a graph: Dynamic graphs often face scenarios where nodes and edges dynamically join or leave the network. To handle these changes effectively, adaptive algorithms that can update the graph's structure in real time are necessary. Incremental algorithms for community detection and centrality measures [57], [213], which adjust the network's representation without requiring a complete re-computation, are particularly valuable. These methods should be designed to maintain the graph's structural integrity and performance as nodes and edges evolve.

Synchronization between entities when sampling: Ensuring synchronization during sampling is crucial for maintaining temporal coherence of a graph. This requires techniques that can sample dynamic graphs in a way that captures the simultaneous evolution of related entities. Coordinated sampling methods [214], [215] that respect temporal dynamics and dependencies within a graph can improve the accuracy of analyses and model training.

Control strategies upon change detection: Implementing control strategies based on detection results enhances the management of dynamic graphs. Systems should autonomously decide on corrective actions or adaptations to maintain performance and stability, such as re-routing data flows or adjusting configurations [216], [217]. Integrating machine learning models to predict the impact of these changes can also further refine control strategies, ensuring proactive and efficient responses.

Hybrid models: Exploring new methodologies, such as random matrix theory [218], graph neural networks (GNNs) [67], and reinforcement learning [216], can lead to more robust and efficient solutions. Hybrid models combining local and global perspectives [36], incorporating topological and temporal information [208], can provide a comprehensive understanding of network changes.

REFERENCES

- [1] H. Cai, V. W. Zheng, and K. C.-C. Chang, "A comprehensive survey of graph embedding: Problems, techniques, and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1616–1637, Sep. 2018.
- [2] P. Goyal and E. Ferrara, "Graph embedding techniques, applications, and performance: A survey," *Knowl. Based Syst.*, vol. 151, pp. 78–94, 2018.
- [3] C. Xu and T. C. Lee, "Statistical consistency for change point detection and community estimation in time-evolving dynamic networks," *IEEE Trans. Signal Inf. Process. Over Netw.*, vol. 8, pp. 215–227, 2022.
- [4] N. Hwang, J. Xu, S. Chatterjee, and S. Bhattacharyya, "The bethe Hessian and information theoretic approaches for online change-point detection in network data," *Sankhya A*, vol. 84, no. 1, pp. 283–320, Jun. 2022.
- [5] J. Cheng, M. Chen, M. Zhou, S. Gao, C. Liu, and C. Liu, "Overlapping community change-point detection in an evolving network," *IEEE Trans. Big Data*, vol. 6, no. 1, pp. 189–200, Mar. 2020.
- [6] I. Cribben, R. Haraldsdottir, L. Y. Atlas, T. D. Wager, and M. A. Lindquist, "Dynamic connectivity regression: Determining state-related changes in brain connectivity," *Neuroimage*, vol. 61, no. 4, pp. 907–920, 2012.
- [7] C. D. Barros, M. R. Mendonça, A. B. Vieira, and A. Ziviani, "A survey on embedding dynamic graphs," *ACM Comput. Surv.*, vol. 55, no. 1, pp. 1–37, 2021.
- [8] G. Xue, M. Zhong, J. Li, J. Chen, C. Zhai, and R. Kong, "Dynamic network embedding survey," *Neurocomputing*, vol. 472, pp. 212–223, 2022.
- [9] P. Goyal, N. Kamra, X. He, and Y. Liu, "DynGEM: Deep embedding method for dynamic graphs," in *Proc. 3rd Int. Workshop Representation Learn. Graphs (ReLiG)*, IJCAI, 2017.
- [10] S. Asur, S. Parthasarathy, and D. Ucar, "An event-based framework for characterizing the evolutionary behavior of interaction graphs," *ACM Trans. Knowl. Discov. Data*, vol. 3, no. 4, pp. 1–36, 2009.
- [11] S. Huang, Y. Hitti, G. Rabusseau, and R. Rabbany, "Laplacian change point detection for dynamic graphs," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 349–358.
- [12] A.-L. Barabási, N. Gulbahce, and J. Loscalzo, "Network medicine: A network-based approach to human disease," *Nat. Rev. Genet.*, vol. 12, no. 1, pp. 56–68, 2011.
- [13] A. G. Tartakovsky, B. L. Rozovskii, R. B. Blazek, and H. Kim, "A novel approach to detection of intrusions in computer networks via adaptive sequential and batch-sequential change-point detection methods," *IEEE Trans. Signal Process.*, vol. 54, no. 9, pp. 3372–3382, Sep. 2006.
- [14] D. Wang, Z. Chen, Y. Fu, Y. Liu, and H. Chen, "Incremental causal graph learning for online root cause analysis," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 2269–2278.
- [15] H. Chen and N. Zhang, "Graph-based change-point detection," *Ann. Statist.*, vol. 43, no. 1, pp. 139–176, 2015.
- [16] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
- [17] J. Liu, D. Yang, K. Zhang, H. Gao, and J. Li, "Anomaly and change point detection for time series with concept drift," *World Wide Web*, vol. 26, no. 5, pp. 3229–3252, 2023.
- [18] M. A. Pimentel, D. A. Clifton, L. Clifton, and L. Tarassenko, "A review of novelty detection," *Signal Process.*, vol. 99, pp. 215–249, 2014.
- [19] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, 2014.
- [20] S. Bhatia, A. Jain, S. Srivastava, K. Kawaguchi, and B. Hooi, "Memstream: Memory-based streaming anomaly detection," in *Proc. ACM Web Conf.*, 2022, pp. 610–621.
- [21] R. Pasricha, E. Gujral, and E. E. Papalexakis, "Identifying and alleviating concept drift in streaming tensor decomposition," in *Proc. Eur. Conf. Mach. Learn. Knowl. Discov. Databases*, Dublin, Ireland, 2019, pp. 327–343.
- [22] D. M. V. Sato, S. C. De Freitas, J. P. Barddal, and E. E. Scalabrin, "A survey on concept drift in process mining," *ACM Comput. Surv.*, vol. 54, no. 9, pp. 1–38, 2021.
- [23] R. G. de Sousa, A. C. M. Neto, M. Fantinato, S. M. Peres, and H. A. Reijers, "Integrated detection and localization of concept drifts in process mining with batch and stream trace clustering support," *Data Knowl. Eng.*, vol. 149, 2024, Art. no. 102253.
- [24] J. Chen, S. Yang, T. Gao, Y. Ying, T. Li, and P. Li, "Multi-type concept drift detection under a dual-layer variable sliding window in frequent pattern mining with cloud computing," *J. Cloud Comput.*, vol. 13, no. 1, 2024, Art. no. 40.
- [25] S. Aminikhanghahi and D. J. Cook, "A survey of methods for time series change point detection," *Knowl. Inf. Syst.*, vol. 51, no. 2, pp. 339–367, 2017.
- [26] M. Han, Z. Chen, M. Li, H. Wu, and X. Zhang, "A survey of active and passive concept drift handling methods," *Comput. Intell.*, vol. 38, no. 4, pp. 1492–1535, 2022.
- [27] C. Truong, L. Oudre, and N. Vayatis, "Selective review of offline change point detection methods," *Signal Process.*, vol. 167, 2020, Art. no. 107299.
- [28] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–38, 2021.
- [29] Z. Tan, A. Jamdagni, X. He, P. Nanda, and R. P. Liu, "Denial-of-service attack detection based on multivariate correlation analysis," in *Proc. 18th Int. Conf. Neural Inf. Process.*, Shanghai, China, 2011, pp. 756–765.
- [30] M. Jedh, L. B. Othmane, N. Ahmed, and B. Bhargava, "Detection of message injection attacks onto the can bus using similarities of successive messages-sequence graphs," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4133–4146, 2021.
- [31] G. Duan, H. Lv, H. Wang, and G. Feng, "Application of a dynamic line graph neural network for intrusion detection with semisupervised learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 699–714, 2022.
- [32] M. Canizo, I. Triguero, A. Conde, and E. Onieva, "Multi-head CNN-RNN for multi-time series anomaly detection: An industrial case study," *Neurocomputing*, vol. 363, pp. 246–260, 2019.
- [33] S.-O. Jeong, C. Pae, and H.-J. Park, "Connectivity-based change point detection for large-size functional networks," *Neuroimage*, vol. 143, pp. 353–363, 2016.
- [34] Y. Lin et al., "Learning dynamic graph embeddings for accurate detection of cognitive state changes in functional brain networks," *NeuroImage*, vol. 230, 2021, Art. no. 117791.
- [35] L. Bian, T. Cui, B. T. Yeo, A. Fornito, A. Razi, and J. Keith, "Identification of community structure-based brain states and transitions using functional MRI," *NeuroImage*, vol. 244, 2021, Art. no. 118635.
- [36] Y. Xie, W. Wang, M. Shao, T. Li, and Y. Yu, "Multi-view change point detection in dynamic networks," *Inf. Sci.*, vol. 629, pp. 344–357, 2023.
- [37] Q. Liu, R. Zhang, and Y. Xie, "Distributed change detection via average consensus over networks," in *Stochastic Models, Statistics and Their Applications*. Dresden, Germany: Springer, 2019, pp. 177–192.
- [38] Z. Sun, S. Zou, R. Zhang, and Q. Li, "Quickest change detection in anonymous heterogeneous sensor networks," *IEEE Trans. Signal Process.*, vol. 70, pp. 1041–1055, 2022.
- [39] S. M. Erfani, S. Rajasegarar, S. Karunasekera, and C. Leckie, "High-dimensional and large-scale anomaly detection using a linear one-class SVM with deep learning," *Pattern Recognit.*, vol. 58, pp. 121–134, 2016.
- [40] L. Zhang, H. Wang, C. Li, Y. Shao, and Q. Ye, "Unsupervised anomaly detection algorithm of graph data based on graph kernel," in *Proc. IEEE 4th Int. Conf. Cyber Secur. Cloud Comput.*, 2017, pp. 58–63.
- [41] R. Zhang, Y. Hao, D. Yu, W.-C. Chang, G. Lai, and Y. Yang, "Correlation-aware change-point detection via graph neural networks," in *Proc. 27th Int. Conf. Neural Inf. Process.*, Bangkok, Thailand, 2020, pp. 555–567.
- [42] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," in *Proc. 2020 IEEE Int. Conf. Data Mining*, 2020, pp. 841–850.
- [43] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.
- [44] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng, "MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks," in *Proc. Int. Conf. Artif. Neural Netw.*, 2019, pp. 703–716.
- [45] Y. Park, C. E. Priebe, and A. Youssef, "Anomaly detection in time series of graphs using fusion of graph invariants," *IEEE J. Sel. Top. Signal Process.*, vol. 7, no. 1, pp. 67–75, Feb. 2013.
- [46] M. Tang, Y. Park, N. H. Lee, and C. E. Priebe, "Attribute fusion in a latent process model for time series of graphs," *IEEE Trans. Signal Process.*, vol. 61, no. 7, pp. 1721–1732, Apr. 2013.
- [47] H. Wang, M. Tang, Y. Park, and C. E. Priebe, "Locality statistics for anomaly detection in time series of graphs," *IEEE Trans. Signal Process.*, vol. 62, no. 3, pp. 703–717, Feb. 2014.
- [48] L. Akoglu and C. Faloutsos, "Event detection in time series of mobile communication graphs," in *Proc. Army Sci. Conf.*, 2010, Art. no. 141.
- [49] H. Hazrati-Marangaloo and R. Noorossana, "A nonparametric change detection approach in social networks," *Qual. Reliab. Eng. Int.*, vol. 37, no. 6, pp. 2916–2935, 2021.

- [50] Y. Chen, X. Mao, D. Ling, and Y. Gu, "Change-point detection of Gaussian graph signals with partial information," in *Proc. 2018 IEEE Int. Conf. Acoust. Speech Signal Process.*, 2018, pp. 3934–3938.
- [51] J. Sharpnack, A. Singh, and A. Rinaldo, "Changepoint detection over graphs with the spectral scan statistic," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2013, pp. 545–553.
- [52] Y. Wang, A. Chakrabarti, D. Sivakoff, and S. Parthasarathy, "Fast change point detection on dynamic social networks," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, 2017, pp. 2992–2998.
- [53] L. Peel and A. Clauset, "Detecting change points in the large-scale structure of evolving networks," in *Proc. AAAI Conf. Artif. Intell.*, 2015, pp. 2914–2920.
- [54] J. D. Wilson, N. T. Stevens, and W. H. Woodall, "Modeling and detecting change in temporal networks via the degree corrected stochastic block model," *Qual. Reliab. Eng. Int.*, vol. 35, no. 5, pp. 1363–1378, 2019.
- [55] O. H. M. Padilla, Y. Yu, and C. E. Priebe, "Change point localization in dependent dynamic nonparametric random dot product graphs," *J. Mach. Learn. Res.*, vol. 23, no. 234, pp. 1–59, 2022.
- [56] B. Marengo, P. Bermolen, M. Fiori, F. Larroca, and G. Mateos, "Online change point detection for weighted and directed random dot product graphs," *IEEE Trans. Signal Inf. Process. Over Netw.*, vol. 8, pp. 144–159, 2022.
- [57] W. Yu, W. Cheng, C. C. Aggarwal, K. Zhang, H. Chen, and W. Wang, "NetWalk: A flexible deep embedding approach for anomaly detection in dynamic networks," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2672–2681.
- [58] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *J. Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.
- [59] Z. Chen, W. Hendrix, and N. F. Samatova, "Community-based anomaly detection in evolutionary networks," *J. Intell. Inf. Syst.*, vol. 39, no. 1, pp. 59–85, 2012.
- [60] D. Duan, Y. Li, Y. Jin, and Z. Lu, "Community mining on dynamic weighted directed graphs," in *Proc. 1st ACM Int. Workshop Complex Netw. Meet Inf. Knowl. Manage.*, 2009, pp. 11–18.
- [61] D. Chakrabarti, "Autopart: Parameter-free graph partitioning and outlier detection," in *Proc. Eur. Conf. Princ. Data Mining Knowl. Discov.*, 2004, pp. 112–124.
- [62] J. Sun, C. Faloutsos, S. Papadimitriou, and P. S. Yu, "GraphScope: Parameter-free mining of large time-evolving graphs," in *Proc. 13th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2007, pp. 687–696.
- [63] C. Xu and T. C. Lee, "Change point detection and node clustering for time series of graphs," *IEEE Trans. Signal Process.*, vol. 70, pp. 3165–3180, 2022.
- [64] T. Gao, W. Peng, D. Sisodia, T. K. Saha, F. Li, and M. Al Hasan, "Android malware detection via graphlet sampling," *IEEE Trans. Mob. Comput.*, vol. 18, no. 12, pp. 2754–2767, Dec. 2019.
- [65] D. Zambon, C. Alippi, and L. Livi, "Change-point methods on a sequence of graphs," *IEEE Trans. Signal Process.*, vol. 67, no. 24, pp. 6327–6341, Dec. 2019.
- [66] J. Li, H. Dani, X. Hu, and H. Liu, "Radar: Residual analysis for anomaly detection in attributed networks," in *Proc. Int. Joint Conf. Artif. Intell.*, vol. 17, 2017, pp. 2152–2158.
- [67] Z. Peng et al., "ANOMALOUS: A joint modeling approach for anomaly detection on attributed networks," in *Proc. Int. Joint Conf. Artif. Intell.*, 2018, pp. 3513–3519.
- [68] A. G. Tartakovsky, B. L. Rozovskii, and K. Shah, "A nonparametric multichart CUSUM test for rapid intrusion detection," in *Proc. Joint Statist. Meetings*, 2005, Art. no. 11.
- [69] A. Lakhina, M. Crovella, and C. Diot, "Diagnosing network-wide traffic anomalies," *SIGCOMM Comput. Commun. Rev.*, vol. 34, no. 4, pp. 219–230, 2004.
- [70] H. H. Pajouh, R. Javidan, R. Khayami, A. Dehghantaha, and K.-K. R. Choo, "A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in IoT backbone networks," *IEEE Trans. Emerg. Top. Comput.*, vol. 7, no. 2, pp. 314–323, Second Quarter, 2019.
- [71] Y. Cao, A. Thompson, M. Wang, and Y. Xie, "Sketching for sequential change-point detection," *EURASIP J. Adv. Signal Process.*, vol. 2019, pp. 1–22, 2019.
- [72] Z. Tan, A. Jamdagni, X. He, P. Nanda, and R. P. Liu, "A system for denial-of-service attack detection based on multivariate correlation analysis," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 2, pp. 447–456, Feb. 2014.
- [73] Z. Tan, A. Jamdagni, X. He, P. Nanda, R. P. Liu, and J. Hu, "Detection of denial-of-service attacks based on computer vision techniques," *IEEE Trans. Comput.*, vol. 64, no. 9, pp. 2519–2533, Sep. 2015.
- [74] I. Barnett and J.-P. Onnela, "Change point detection in correlation networks," *Sci. Rep.*, vol. 6, no. 1, 2016, Art. no. 18893.
- [75] A. Lakhina, M. Crovella, and C. Diot, "Mining anomalies using traffic feature distributions," *SIGCOMM Comput. Commun. Rev.*, vol. 35, no. 4, pp. 217–228, 2005.
- [76] P. Berezinski, B. Jasiul, and M. Szyrka, "An entropy-based network anomaly detection method," *Entropy*, vol. 17, no. 4, pp. 2367–2408, 2015.
- [77] M. Petkovic, I. Basicovic, D. Kukolj, and M. Popović, "Evaluation of Takagi-Sugeno-Kang fuzzy method in entropy-based detection of DDoS attacks," *Comput. Sci. Inf. Syst.*, vol. 15, no. 1, pp. 139–162, 2018.
- [78] S. Garg, K. Kaur, N. Kumar, and J. J. Rodrigues, "Hybrid deep-learning-based anomaly detection scheme for suspicious flow detection in SDN: A social multimedia perspective," *IEEE Trans. Multimed.*, vol. 21, no. 3, pp. 566–578, Mar. 2019.
- [79] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD*, 2019, pp. 2828–2837.
- [80] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3395–3404.
- [81] O. Salem, S. Vaton, and A. Gravey, "A scalable, efficient and informative approach for anomaly-based intrusion detection systems: Theory and practice," *Int. J. Netw. Manag.*, vol. 20, no. 5, pp. 271–293, 2010.
- [82] D. Zambon, L. Livi, and C. Alippi, "Detecting changes in sequences of attributed graphs," in *Proc. 2017 IEEE Symp. Ser. Comput. Intell.*, 2017, pp. 1–7.
- [83] R. P. Adams and D. J. MacKay, "Bayesian online changepoint detection," 2007, *arXiv:0710.3742*.
- [84] T. Zhu, P. Li, L. Yu, K. Chen, and Y. Chen, "Change point detection in dynamic networks based on community identification," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 3, pp. 2067–2077, Jul.-Sep. 2020.
- [85] Y.-W. Liu and H. Chen, "A fast and efficient change-point detection framework based on approximate K-nearest neighbor graphs," *IEEE Trans. Signal Process.*, vol. 70, pp. 1976–1986, 2022.
- [86] A. A. Cook, G. Misirlı, and Z. Fan, "Anomaly detection for IoT time-series data: A survey," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6481–6494, Jul. 2020.
- [87] P. Cui, X. Wang, J. Pei, and W. Zhu, "A survey on network embedding," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 5, pp. 833–852, May 2019.
- [88] D. Zhang, J. Yin, X. Zhu, and C. Zhang, "Network representation learning: A survey," *IEEE Trans. Big Data*, vol. 6, no. 1, pp. 3–28, Mar. 2020.
- [89] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: A survey," *Data Min. Knowl. Discov.*, vol. 29, pp. 626–688, 2015.
- [90] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023.
- [91] S. Ranshous, S. Shen, D. Koutra, S. Harenberg, C. Faloutsos, and N. F. Samatova, "Anomaly detection in dynamic networks: A survey," *Wiley Interdiscipl. Rev.: Comput. Statist.*, vol. 7, no. 3, pp. 223–247, 2015.
- [92] D. Savage, X. Zhang, X. Yu, P. Chou, and Q. Wang, "Anomaly detection in online social networks," *Soc. Netw.*, vol. 39, pp. 62–70, 2014.
- [93] A. D. Pazho, G. A. Noghre, A. A. Purkayastha, J. Vempati, O. Martin, and H. Tabkhi, "A survey of graph-based deep learning for anomaly detection in distributed systems," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 1, pp. 1–20, Jan. 2024.
- [94] M. Jin et al., "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10466–10485, Dec. 2024.
- [95] H. Wang, M. J. Bah, and M. Hammad, "Progress in outlier detection techniques: A survey," *IEEE Access*, vol. 7, pp. 107964–108000, 2019.
- [96] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1035–1044.
- [97] Z. Tao, Y. Wei, X. Wang, X. He, X. Huang, and T.-S. Chua, "Mgat: Multimodal graph attention network for recommendation," *Inf. Process. Manag.*, vol. 57, no. 5, 2020, Art. no. 102277.

- [98] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. 2009 IEEE Symp. Comput. Intell. Secur. Defense Appl.*, 2009, pp. 1–6.
- [99] G. Wang, J. Yang, and R. Li, "Imbalanced SVM-Based Anomaly Detection Algorithm for Imbalanced Training Datasets," in *Etri J.*, vol. 39, no. 5, pp. 621–631, 2017, Wiley Online Library.
- [100] S. Niu, Y. Liu, J. Wang, and H. Song, "A decade survey of transfer learning (2010–2020)," *IEEE Trans. Artif. Intell.*, vol. 1, no. 2, pp. 151–166, Oct. 2020.
- [101] R. Caruana, N. Karampatziakis, and A. Yessenalina, "An empirical evaluation of supervised learning in high dimensions," in *Proc. 25th Int. Conf. Mach. Learn.*, 2008, pp. 96–103.
- [102] E. Arazo, D. Ortego, P. Albert, N. E. O'Connor, and K. McGuinness, "Pseudo-labeling and confirmation bias in deep semi-supervised learning," in *Proc. 2020 Int. Joint Conf. Neural Netw.*, 2020, pp. 1–8.
- [103] P. Cascante-Bonilla, F. Tan, Y. Qi, and V. Ordóñez, "Curriculum labeling: Revisiting pseudo-labeling for semi-supervised learning," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 6912–6920.
- [104] S. Baek, D. Kwon, J. Kim, S. C. Suh, H. Kim, and I. Kim, "Unsupervised labeling for supervised anomaly detection in enterprise and cloud networks," in *Proc. 2017 IEEE 4th Int. Conf. Cyber Secur. Cloud Comput.*, 2017, pp. 205–210.
- [105] E.-S. Apostol, C.-O. Truică, F. Pop, and C. Esposito, "Change point enhanced anomaly detection for IoT time series data," *Water*, vol. 13, no. 12, 2021, Art. no. 1633.
- [106] X. Tao, Y. Peng, F. Zhao, S. Wang, and Z. Liu, "An improved parallel network traffic anomaly detection method based on bagging and GRU," in *Proc. Int. Conf. Wirel. Algorithms, Syst., Appl.*, Springer, 2020, pp. 420–431.
- [107] S. Fakhræi, J. Foulds, M. Shashanka, and L. Getoor, "Collective spammer detection in evolving multi-relational social networks," in *Proc. 21th ACM SIGKDD*, 2015, pp. 1769–1778.
- [108] M. Hu, G. Xu, C. Ma, and M. Daneshmand, "Detecting review spammer groups in dynamic review networks," in *Proc. ACM Turing Celebration Conf.-China*, 2019, pp. 1–6.
- [109] Y. Cao, H. Jiang, Y. Deng, J. Wu, P. Zhou, and W. Luo, "Detecting and mitigating DDoS attacks in SDN using spatial-temporal graph convolutional network," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 6, pp. 3855–3872, Nov./Dec. 2022.
- [110] I. Cribben, T. D. Wager, and M. A. Lindquist, "Detecting functional connectivity change points for single-subject fMRI data," in *Front. Comput. Neurosci.*, vol. 7, 2013, Art. no. 143, Frontiers Media SA.
- [111] C. K. Dagli and W. M. Campbell, "Individual and group dynamics in the reality mining corpus," in *Proc. 2012 Int. Conf. Privacy, Secur., Risk Trust 2012 Int. Conf. Social Comput.*, 2012, pp. 61–70.
- [112] N. Eagle and A. Pentland, "Reality mining: Sensing complex social systems," in *Pers. Ubiquitous Comput.*, vol. 10, pp. 255–268, 2006, Springer.
- [113] G. E. Hinton, T. J. Sejnowski, and D. H. Ackley, "Boltzmann machines: Constraint satisfaction networks that learn," Carnegie Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU-CS-84-119, 1984.
- [114] Y. Mei, "Information bounds and quickest change detection in decentralized decision systems," *IEEE Trans. Inf. Theory*, vol. 51, no. 7, pp. 2669–2681, Jul. 2005.
- [115] A. G. Tartakovsky and V. V. Veeravalli, "Asymptotically optimal quickest change detection in distributed sensor systems," *Sequential Anal.*, vol. 27, no. 4, pp. 441–475, 2008.
- [116] R. Olfati-Saber, J. A. Fax, and R. M. Murray, "Consensus and cooperation in networked multi-agent systems," in *Proc. IEEE*, vol. 95, no. 1, pp. 215–233, Jan. 2007.
- [117] P. Braca, S. Marano, V. Matta, and P. Willett, "Consensus-based Page's test in sensor networks," *Signal Process.*, vol. 91, no. 4, pp. 919–930, 2011.
- [118] Y. Gu, Y. Jiao, X. Xu, and Q. Yu, "Request–response and censoring-based energy-efficient decentralized change-point detection with IoT applications," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6771–6788, Apr. 2021.
- [119] M. Nigro, S. N. Pakzad, and S. Dorvash, "Localized structural damage detection: A change point analysis," *Comput.-Aided Civil Infrastructure Eng.*, vol. 29, no. 6, pp. 416–432, 2014.
- [120] R. An, C. Yang, and Y. Pan, "Unsupervised change point detection using a weight graph method for process monitoring," *Ind. Eng. Chem. Res.*, vol. 58, no. 4, pp. 1624–1634, 2019.
- [121] E. García, N. Montés, J. Llopis, and A. Lacasa, "Evaluation of change point detection algorithms for application in big data mini-term 4.0," in *Proc. 17th Int. Conf. Inform. Control, Autom. Robot.*, 2020, pp. 117–124.
- [122] Z. Abbas, P. Sottovia, M. A. H. Hassan, D. Foroni, and S. Bortoli, "Real-time traffic JAM detection and congestion reduction using streaming graph analytics," in *Proc. 2020 IEEE Int. Conf. Big Data*, 2020, pp. 3109–3118.
- [123] Y. Guo, L. Yang, S. Hao, and J. Gao, "Dynamic identification of urban traffic congestion warning communities in heterogeneous networks," *Physica A: Statist. Mech. Appl.*, vol. 522, pp. 98–111, 2019.
- [124] S. R. Cachay et al., "The world as a graph: Improving El NINO forecasts with graph neural networks," 2021, *arXiv:2104.05089*.
- [125] Y. Chen, N. Crespi, A. M. Ortiz, and L. Shu, "Reality mining: A prediction algorithm for disease dynamics based on mobile big data," *Inf. Sci.*, vol. 379, pp. 82–93, 2017.
- [126] Y. Chen, L. Zhao, X. Peng, J. Yuan, and D. N. Metaxas, "Construct dynamic graphs for hand gesture recognition via spatial-temporal attention," 2019, *arXiv: 1907.08871*.
- [127] B. Wang, J. M. Phillips, R. Schreiber, D. Wilkinson, N. Mishra, and R. Tarjan, "Spatial scan statistics for graph clustering," in *Proc. 2008 SIAM Int. Conf. Data Mining*, 2008, pp. 727–738.
- [128] P. Erdős and A. Rényi, "On the evolution of random graphs," *Pub. Math. Inst. Hung. Acad. Sci.*, vol. 5, pp. 17–61, 1960.
- [129] P. W. Holland, K. B. Laskey, and S. Leinhardt, "Stochastic blockmodels: First steps," *Soc. Netw.*, vol. 5, no. 2, pp. 109–137, 1983.
- [130] P. D. Hoff, A. E. Raftery, and M. S. Handcock, "Latent space approaches to social network analysis," *J. Amer. Statist. Assoc.*, vol. 97, no. 460, pp. 1090–1098, 2002.
- [131] S. J. Young and E. R. Scheinerman, "Random dot product graph models for social networks," in *Proc. Int. Workshop Algorithms Models Web-Graph*, 2007, pp. 138–149.
- [132] Z. Zhao, L. Chen, and L. Lin, "Change-point detection in dynamic networks via graphon estimation," 2019, *arXiv: 1908.01823*.
- [133] D. Wang, Y. Yu, and A. Rinaldo, "Optimal change point detection and localization in sparse dynamic networks," *Ann. Statist.*, vol. 49, no. 1, pp. 203–232, 2021.
- [134] W. H. Mow, "Maximum likelihood sequence estimation from the lattice viewpoint," *IEEE Trans. Inf. Theory*, vol. 40, no. 5, pp. 1591–1600, Sep. 1994.
- [135] A. Grover and J. Leskovec, "Node2Vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.
- [136] D. Eswaran, C. Faloutsos, S. Guha, and N. Mishra, "Spotlight: Detecting anomalies in streaming graphs," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 1378–1386.
- [137] S. Dongen, *A Cluster Algorithm for Graphs*. Nampa, ID, USA: CWI (Centre for Mathematics and Computer Science), 2000.
- [138] M. E. Newman, "Fast algorithm for detecting community structure in networks," *Phys. Rev. E—Stat., Nonlinear, Soft Matter Phys.*, vol. 69, no. 6, 2004, Art. no. 066133.
- [139] J. Rissanen, "Modeling by shortest data description," *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.
- [140] P. Ribeiro, P. Paredes, M. E. Silva, D. Aparicio, and F. Silva, "A survey on subgraph counting: Concepts, algorithms, and applications to network motifs and graphlets," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–36, 2021.
- [141] B. J. Jain, "On the geometry of graph spaces," *Discrete Appl. Math.*, vol. 214, pp. 126–144, 2016.
- [142] R. P. Duin and E. Pekalska, "Non-euclidean dissimilarities: Causes and informativeness," in *Proc. Struct., Syntactic, Statist. Pattern Recognit.: Joint IAPR Int. Workshop*, Springer, 2010, pp. 324–333.
- [143] M. Lichter, S. Raßmann, and P. Schweitzer, "Computational complexity of the Weisfeiler-Leman dimension," 2024, *arXiv:2402.11531*.
- [144] M. W. Mahoney and P. Drineas, "CUR matrix decompositions for improved data analysis," in *Proc. Nat. Acad. Sci. USA*, vol. 106, no. 3, pp. 697–702, 2009.
- [145] E. S. Page, "Continuous inspection schemes," *Biométrica*, vol. 41, no. 1/2, pp. 100–115, 1954.
- [146] P. Granjon, "The CuSum algorithm—A small review," 2013. Accessed: Jan. 6, 2025. [Online]. Available: <https://hal.archives-ouvertes.fr/hal-00914697>
- [147] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Ann. Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.
- [148] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.
- [149] C. Tsallis, "Possible generalization of Boltzmann-Gibbs statistics," *J. Statist. Phys.*, vol. 52, pp. 479–487, 1988.

- [150] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal convolutional networks for action segmentation and detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 156–165.
- [151] H. Han et al., "STGCN: A spatial-temporal aware graph learning method for POI recommendation," in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 1052–1057.
- [152] L.-R. Yu, Q.-H. Lu, and Y. Xue, "DTAAD: Dual TCN-attention networks for anomaly detection in multivariate time series data," *Knowl. Based Syst.*, vol. 295, 2024, Art. no. 111849.
- [153] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by error propagation," *Parallel Distributed Processing, Explorations in the Microstructure of Cognition*, vol. 1, 1986, pp. 318–362.
- [154] A. Vaswani, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [155] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jan. 2021.
- [156] Q. Wen et al., "Transformers in time series: A survey," in *Proc. 32nd Int. Joint Conf. Artif. Intell.*, 2023, pp. 6778–6786.
- [157] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [158] C. Ding, S. Sun, and J. Zhao, "MST-GAT: A multimodal spatial-temporal graph attention network for time series anomaly detection," *Inf. Fusion*, vol. 89, pp. 527–536, 2023.
- [159] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [160] D. Grattarola, D. Zamboni, L. Livi, and C. Alippi, "Change detection in graph streams by learning graph embeddings on constant-curvature manifolds," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 6, pp. 1856–1869, Jun. 2020.
- [161] T. De Ryck, M. De Vos, and A. Bertrand, "Change point detection in time series data using autoencoders with a time-invariant representation," *IEEE Trans. Signal Process.*, vol. 69, pp. 3513–3524, 2021.
- [162] M. Gupta, R. Wadhvani, and A. Rasool, "Real-time change-point detection: A deep neural network-based adaptive approach for detecting changes in multivariate time series data," *Expert Syst. Appl.*, vol. 209, 2022, Art. no. 118260.
- [163] C. C. Aggarwal, Y. Li, and P. S. Yu, "On supervised change detection in graph streams," in *Proc. 2020 SIAM Int. Conf. Data Mining*, 2020, pp. 289–297.
- [164] J. Tang, J. Zhang, L. Yao, J. Li, L. Zhang, and Z. Su, "ArnetMiner: Extraction and mining of academic social networks," in *Proc. 14th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2008, pp. 990–998.
- [165] L. Chu and H. Chen, "Sequential change-point detection for high-dimensional and non-Euclidean data," *IEEE Trans. Signal Process.*, vol. 70, pp. 4498–4511, 2022.
- [166] C. M. Ahmed and N. K. Kandasamy, "A comprehensive dataset from a smart grid testbed for machine learning based CPS security research," in *Proc. Cyber-Phys. Secur. Crit. Infrastructures Protection: First Int. Workshop, CPS4CIP 2020, Guildford, UK, September 18, 2020, Revised Sel. Papers 1*, Springer, 2021, pp. 123–135.
- [167] S. Bandyopadhyay, N. Lokesh, and M. N. Murty, "Outlier aware network embedding for attributed networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 12–19.
- [168] J. Leskovec, J. Kleinberg, and C. Faloutsos, "Graph evolution: Densification and shrinking diameters," *ACM Trans. Knowl. Discov. Data*, vol. 1, no. 1, pp. 2–es, 2007.
- [169] K. D. Bollacker, S. Lawrence, and C. L. Giles, "CiteSeer: An autonomous web agent for automatic retrieval and identification of interesting publications," in *Proc. 2nd Int. Conf. Auton. Agents*, 1998, pp. 116–123.
- [170] P. Das, B. P. R. Guda, S. B. Seelaboyina, S. Sarkar, and A. Mukherjee, "Quality change: Norm or exception? Measurement, analysis and detection of quality change in Wikipedia," in *Proc. ACM Hum.-Comput. Inter.*, vol. 6, no. CSCW1, pp. 1–36, 2022.
- [171] Q. Li, M. Cheng, J. Wang, and B. Sun, "LSTM based phishing detection for big email data," *IEEE Trans. Big Data*, vol. 8, no. 1, pp. 278–288, Feb. 2022.
- [172] D. Homscheid, J. Kunegis, and M. Schaarschmidt, "Private-collective innovation and open source software: Longitudinal insights from linux kernel development," in *Proc. Open Big Data Manage. Innov.: 14th IFIP WG 6.11 Conf. E-Bus., E-Serv., E-Soc., I3E 2015, Delft, The Netherlands, October 13–15, 2015*, Springer, 2015, pp. 299–313.
- [173] A. Foroozani and M. Ebrahimi, "Anomalous information diffusion in social networks: Twitter and Digg," *Expert Syst. Appl.*, vol. 134, pp. 249–266, 2019.
- [174] Y. Kim and K. Shim, "HotDigg: Finding recent hot topics from Digg," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, Springer, 2012, pp. 414–427.
- [175] S. K. Tasoulis, A. G. Vrahatis, S. V. Georgakopoulos, and V. P. Plagianakos, "Real time sentiment change detection of Twitter data streams," in *Proc. 2018 Innov. Intell. Syst. Appl.*, 2018, pp. 1–6.
- [176] Z. Shah and A. G. Dunn, "Event detection on Twitter by mapping unexpected changes in streaming data into a spatiotemporal lattice," *IEEE Trans. Big Data*, vol. 8, no. 2, pp. 508–522, Apr. 2022.
- [177] N. Z. Gong et al., "Evolution of social-attribute networks: Measurements, modeling, and implications using Google+," in *Proc. 2012 Internet Meas. Conf.*, 2012, pp. 131–144.
- [178] R. Gonzalez, R. Cuevas, R. Motamedi, R. Rejaie, and A. Cuevas, "Assessing the evolution of Google+ in its first two years," *IEEE/ACM Trans. Netw.*, vol. 24, no. 3, pp. 1813–1826, Jun. 2016.
- [179] I. U. Hewapathirana, "Change detection in dynamic attributed networks," *WIREs Data Mining Knowl. Discov.*, vol. 9, no. 3, 2019, Art. no. e1286.
- [180] S. R. Bommakanti and S. Panda, "Events detection in temporally evolving social networks," in *Proc. 2018 IEEE Int. Conf. Big Knowl.*, 2018, pp. 235–242.
- [181] H. Su, S. Qin, Z. Wu, T. Peng, S. Liu, and Y. Zhou, "Graph sketch based heavy change detection for temporal network traffic analysis," in *Proc. 2023 Int. Conf. Cyber-Enabled Distrib. Comput. Knowl. Discov.*, 2023, pp. 200–209.
- [182] M. Weber, J. Jost, and E. Saucan, "Forman-Ricci flow for change detection in large dynamic data sets," *Axioms*, vol. 5, no. 4, 2016, Art. no. 26.
- [183] B. Zhang, R. Liu, D. Massey, and L. Zhang, "Collecting the Internet AS-level topology," *SIGCOMM Comput. Commun. Rev.*, vol. 35, no. 1, pp. 53–61, 2005.
- [184] J. G. White et al., "The structure of the nervous system of the nematode *Caenorhabditis elegans*," *Philos. Trans. R Soc. Lond. B Biol. Sci.*, vol. 314, no. 1165, pp. 1–340, 1986.
- [185] Y. Zheng, H. Zhang, and Y. Yu, "Detecting collective anomalies from multiple spatio-temporal datasets across different domains," in *Proc. 23rd SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, 2015, pp. 1–10.
- [186] X. Zhou, "Understanding spatiotemporal patterns of biking behavior by analyzing massive bike sharing data in Chicago," *PLoS One*, vol. 10, no. 10, 2015, Art. no. e0137922.
- [187] B. Hooi et al., "ChangeDAR: Online localized change detection for sensor data on a graph," in *Proc. 27th ACM Int. Conf. Inf. Knowl. Manage.*, 2018, pp. 507–516.
- [188] M. Das and S. Parthasarathy, "Anomaly detection and spatio-temporal analysis of global climate system," in *Proc. 3rd Int. Workshop Knowl. Discov. Sensor Data*, 2009, pp. 142–150.
- [189] L. Lefort et al., "The ACORN-SAT linked climate dataset," *Semantic Web*, vol. 8, no. 6, pp. 959–967, 2017.
- [190] R. Kronenberg, J. Franke, C. Bernhofer, and P. Körner, "Detection of potential areas of changing climatic conditions at a regional scale until 2100 for Saxony, Germany," *Meteorol. Hydrol. Water Manage.*, vol. 3, no. 2, pp. 17–26, 2015.
- [191] C. M. Henry, D. M. Allen, and J. Huang, "Groundwater storage variability and annual recharge using well-hydrograph and GRACE satellite data," *Hydrogeol. J.*, vol. 19, no. 4, 2011, Art. no. 741.
- [192] F. Zihang, N. Boesche, C. Ji, and H. Kaufmann, "Evolution of temperature of a case-II water and an inland lake around the Weihai area, Shandong province between 1994 and 2014," in *Proc. 2016 IEEE Int. Geosci. Remote Sens. Symp.*, 2016, pp. 2316–2319.
- [193] N. Aggarwal, I. Pundir, U. Shukla, and S. Singh, "Spatiotemporal prediction of Chickenpox cases using graph attention network," in *Proc. 2022 IEEE 19th India Council Int. Conf.*, 2022, pp. 1–5.
- [194] Y. Bermudez et al., "Geographic examination of COVID-19 test percent positivity and proportional change in cancer screening volume, national breast and cervical cancer early detection program," *Preventing Chronic Dis.*, vol. 19, 2022, Art. no. E59.
- [195] M. Buscema, M. Asadi-Zeydabadi, W. Lodwick, A. Nde Nembot, A. Bronstein, and F. Newman, "Analysis of the ebola outbreak in 2014 and 2018 in West Africa and Congo by using artificial adaptive systems," *Appl. Artif. Intell.*, vol. 34, no. 8, pp. 597–617, 2020.
- [196] S. Sareen, S. K. Sood, and S. K. Gupta, "IoT-based cloud framework to control Ebola virus outbreak," *J. Ambient Intell. Humanized Comput.*, vol. 9, pp. 459–476, 2018.
- [197] R. L. Rosa et al., "Event detection system based on user behavior changes in online social networks: Case of the COVID-19 pandemic," *IEEE Access*, vol. 8, pp. 158806–158825, 2020.

- [198] S. Montariol, M. Martinc, and L. Pivovarov, "Scalable and interpretable semantic change detection," in *Proc. 2021 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2021, pp. 4642–4652.
- [199] B. Jones, and A. de Sherbinin, *Documentation for the Groundswell Spatial Population and Migration Projections at One-Eight Degree According to SSPs and RCPs, 2010–2050*, Palisades, NY, USA: NASA Socioeconomic Data and Application Centre (SEDAC), 2022.
- [200] G. J. Abel, J. DeWaard, J. T. Ha, and Z. W. Almquist, "The form and evolution of international migration networks, 1990–2015," *Popul. Space Place*, vol. 27, no. 3, 2021, Art. no. e2432.
- [201] M. Zubair, H. Janicke, A. Mohsin, L. Maglaras, and I. H. Sarker, "Automated Sensor Node Malicious Activity Detection with Explainability Analysis," *Sensors*, vol. 24, no. 12, 2024, Art. no. 3712.
- [202] N. Singh, D. Virmani, and X.-Z. Gao, "A fuzzy logic-based method to avert intrusions in wireless sensor networks using WSN-DS dataset," *Int. J. Comput. Intell. Appl.*, vol. 19, no. 03, 2020, Art. no. 2050018.
- [203] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.
- [204] Y. Jiao, X. Xu, and Y. Gu, "Quantization for communication-efficient change-point detection over networks," *IEEE Trans. Signal Process.*, vol. 70, pp. 142–157, 2021.
- [205] J. Ngiam, A. Khosla, M. Kim, J. Nam, H. Lee, and A. Y. Ng, "Multimodal deep learning," in *Proc. 28th Int. Conf. Mach. Learn.*, 2011, pp. 689–696.
- [206] Z. Zhang et al., "Graph neural network approaches for drug-target interactions," *Curr. Opin. Struct. Biol.*, vol. 73, 2022, Art. no. 102327.
- [207] D. Agudelo-España, S. Gomez-Gonzalez, S. Bauer, B. Schölkopf, and J. Peters, "Bayesian online prediction of change points," in *Proc. Conf. Uncertainty Artif. Intell.*, 2020, pp. 320–329.
- [208] M. Wang, L. Wu, M. Li, D. Wu, X. Shi, and C. Ma, "Meta-learning based spatial-temporal graph attention network for traffic signal control," *Knowl. Based Syst.*, vol. 250, 2022, Art. no. 109166.
- [209] B. Huang, C. J. H. Low, F. Xie, C. Glymour, and K. Zhang, "Latent hierarchical causal structure discovery with rank constraints," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 5549–5561.
- [210] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 5363–5370.
- [211] R. Trivedi, H. Dai, Y. Wang, and L. Song, "Know-evolve: Deep temporal reasoning for dynamic knowledge graphs," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3462–3471.
- [212] S. Kumar, X. Zhang, and J. Leskovec, "Predicting dynamic embedding trajectory in temporal interaction networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 1269–1278.
- [213] T. Abughofa, A. A. Harby, H. Isah, and F. Zulkernine, "Incremental community detection in distributed dynamic graph," in *Proc. 2021 IEEE 7th Int. Conf. Big Data Comput. Serv. Appl.*, 2021, pp. 50–59.
- [214] E. Cohen and H. Kaplan, "What you can do with coordinated samples," in *Proc. Int. Workshop Approximation Algorithms Combinatorial Optim.*, 2013, pp. 452–467.
- [215] B. García-Martínez, A. Fernández-Caballero, R. Alcaraz, and A. Martínez-Rodrigo, "Cross-sample entropy for the study of coordinated brain activity in calm and distress conditions with electroencephalographic recordings," *Neural Comput. Appl.*, vol. 33, pp. 9343–9352, 2021.
- [216] Y. Sun, K. Lin, and A. K. Bashir, "KeyLight: Intelligent traffic signal control method based on improved graph neural network," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2861–2871, Feb. 2023.
- [217] D. Xu, P. Liu, H. Li, H. Guo, Z. Xie, and Q. Xuan, "Multi-view graph convolution network reinforcement learning for CAVs cooperative control in highway mixed traffic," *IEEE Trans. Intell. Veh.*, vol. 9, no. 1, pp. 2588–2599, Jan. 2024.
- [218] C. Hoffman, M. Kahle, and E. Paquette, "Spectral gaps of random graphs and applications," *Int. Math. Res. Notices*, vol. 2021, no. 11, pp. 8353–8404, 2021.



Yuxuan Zhou received the BS degree in computer science and technology from the Chongqing University of Posts and Telecommunications, China, in 2022. He is currently working toward the MS degree with Jilin University, China. His research interests include deep learning, reinforcement learning, and cloud computing.



Shang Gao received the BMath degree in computer science from the University of Waterloo, Canada, in 2006, and the MS and PhD degrees in computer science from the University of Calgary, Canada, in 2009 and 2014, respectively. He was an engineer with SAP Labs China from 2006 to 2007. He is now a professor with the College of Computer Science and Technology, Jilin University, China. His research interests include cloud computing, Big Data systems, and optimization.



Dandan Guo received the BS degree from the North University of China, Taiyuan, in 2014, and the PhD degree in electronic engineering from Xidian University, Xi'an, in 2020. She then worked as a postdoctoral researcher with the Chinese University of Hong Kong, Shenzhen. She is currently a tenure-track professor with Jilin University, China. Her research focuses on machine learning algorithms and applications, including radar automatic target recognition, dynamical models, few-shot learning, and imbalanced classification.



Xiaohui Wei received the BS, MS, and PhD degrees in computer science from Jilin University, Changchun, China, in 1993, 1996, and 1999, respectively. He is a professor with the College of Computer Science and Technology, Jilin University, where he is currently the director of the High-Performance Computing Center. His research interests include resource scheduling for large distributed systems, infrastructure-level virtualization, large-scale data processing systems, and fault-tolerant computing.



Jon Rokne (Life Fellow, IEEE) is a professor with the Department of Computer Science, University of Calgary, where he served as a chair from 1989 to 1996. His research interests include interval analysis, global optimization, computer graphics, and social networks. He co-edited the *Encyclopedia of Social Network Analysis and Mining* (Springer 2014, 2nd ed. 2018). He organized the first university FDDI network in Canada with the University of Calgary and worked to connect the University to the WWW, in 1990. He led the installation of a top 500 supercomputing facility in 1992. An IEEE member since 1970 and life fellow since 2013, he has served two terms as vice-president, Publications, and three terms on the Board of Governors of the IEEE Computer Society, as well as vice-president, IEEE Publication Services and Products Board, and on the IEEE Board of Directors for two terms.



Hui Wang received the BSc degree in computer science and the MSc degree in artificial intelligence from Jilin University, China, in 1985 and 1988, respectively, and the DPhil degree in informatics from the University of Ulster, U.K., in 1996. From 1988 to 1992, he was a lecturer with Jilin University. He is currently a professor with the School of Electronics, Electrical Engineering, and Computer Science, Queen's University Belfast, U.K. He has authored or coauthored more than 300 publications. His research interests include machine learning, logics and reasoning, combinatorial data analytics, and their applications in image, video, spectra, and text analysis.